

lectures again

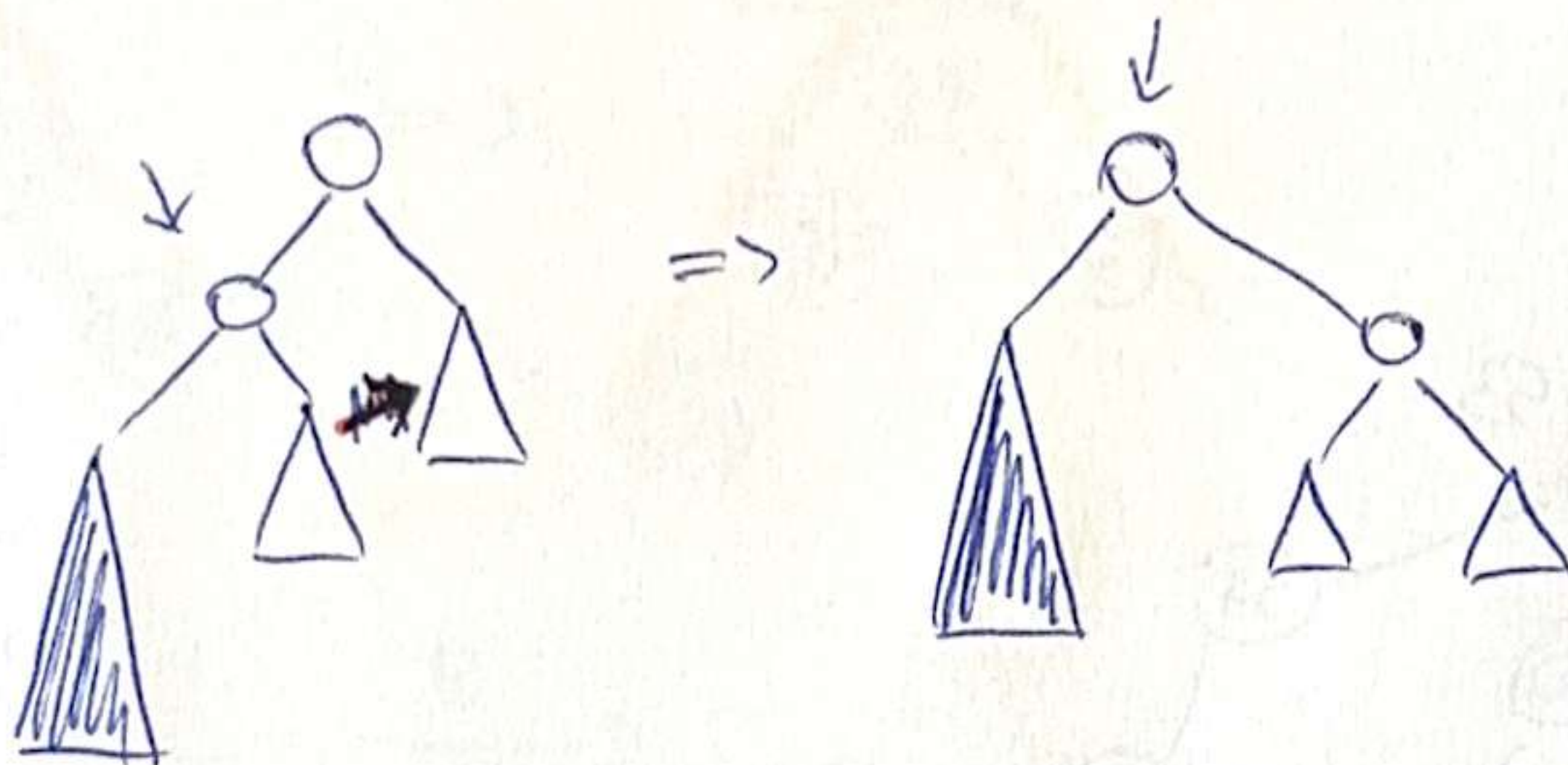
Lecture 14

AVL = BT, $\text{height-left} - \text{height-right} \in \{0, 1, -1\}$.

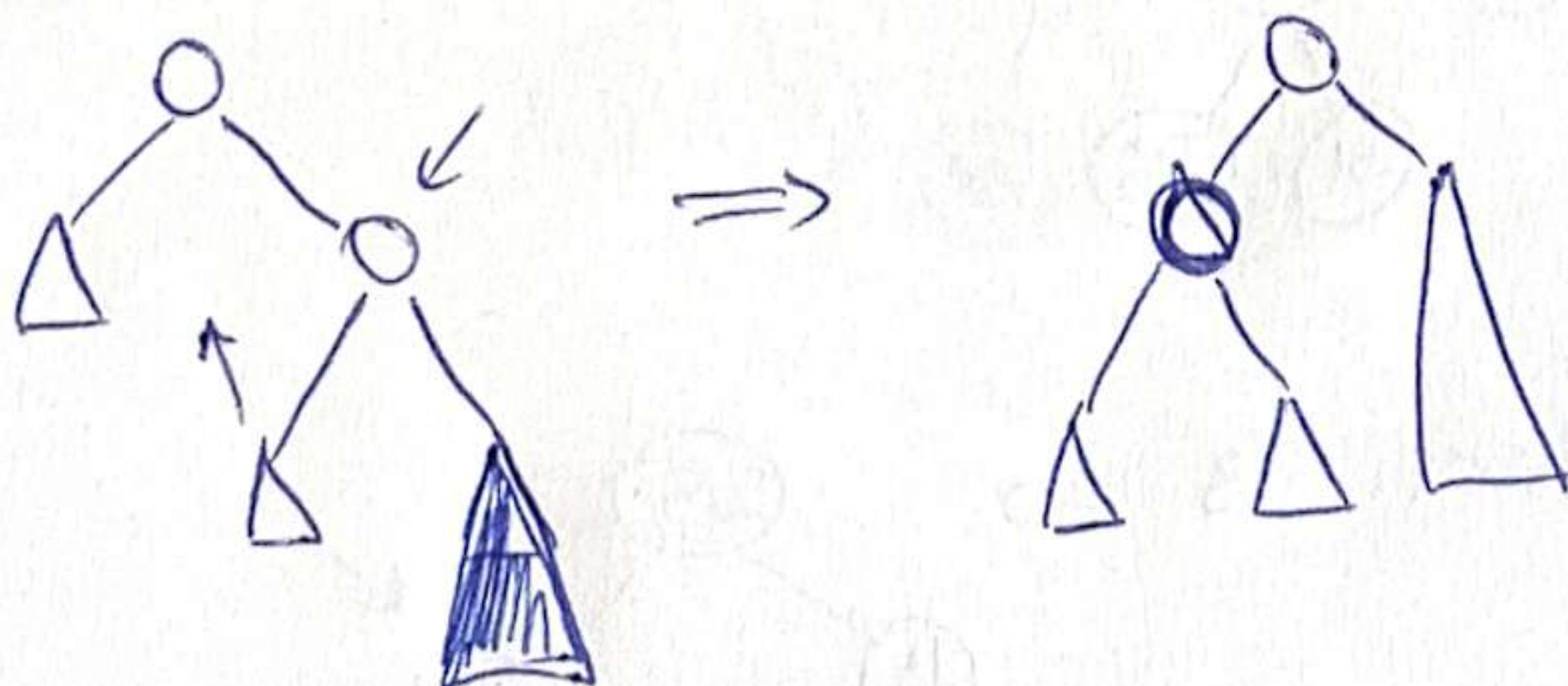
Empty tree $\Rightarrow h = -1$.

Single node $\Rightarrow 0 = h$

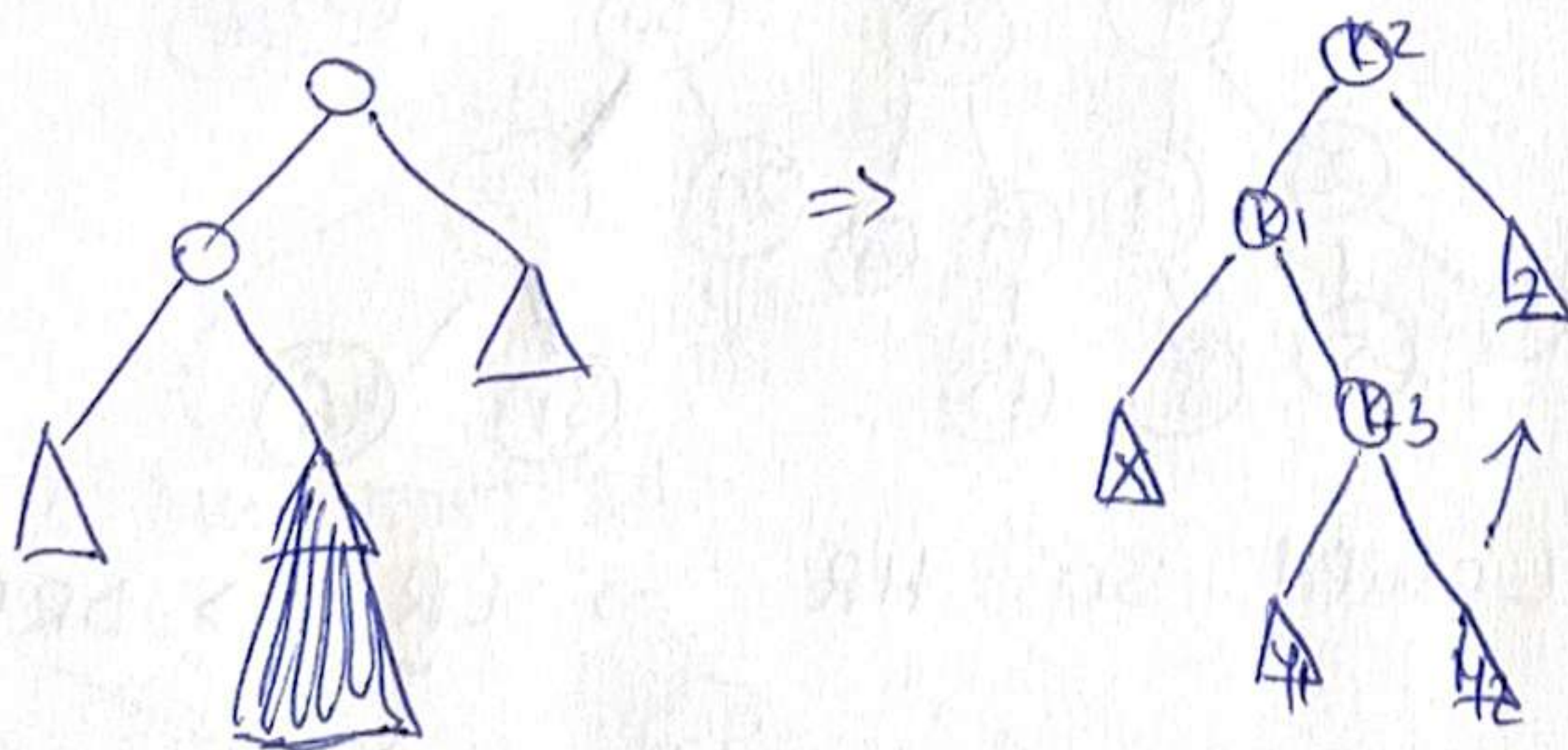
SRR:



SRL:



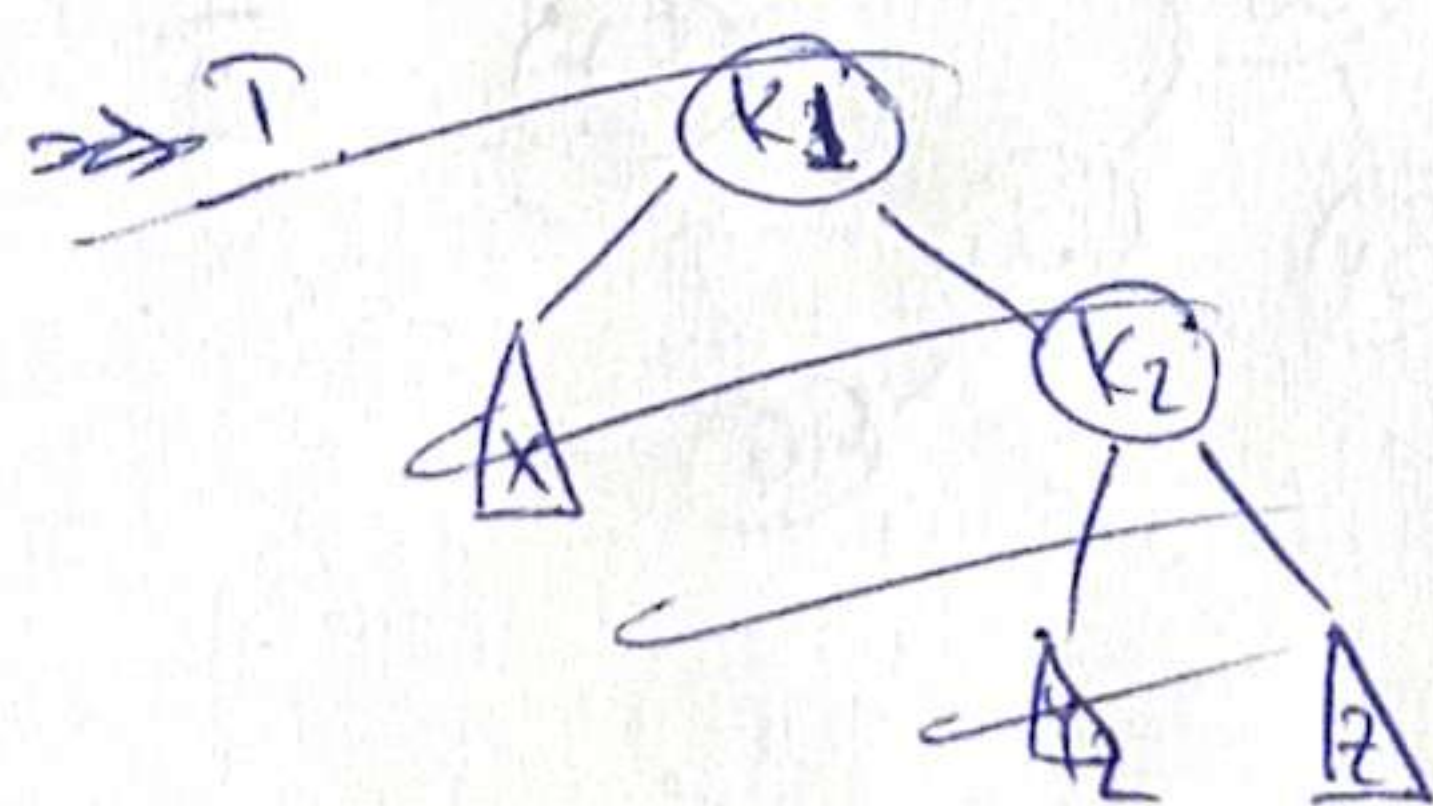
DRR:



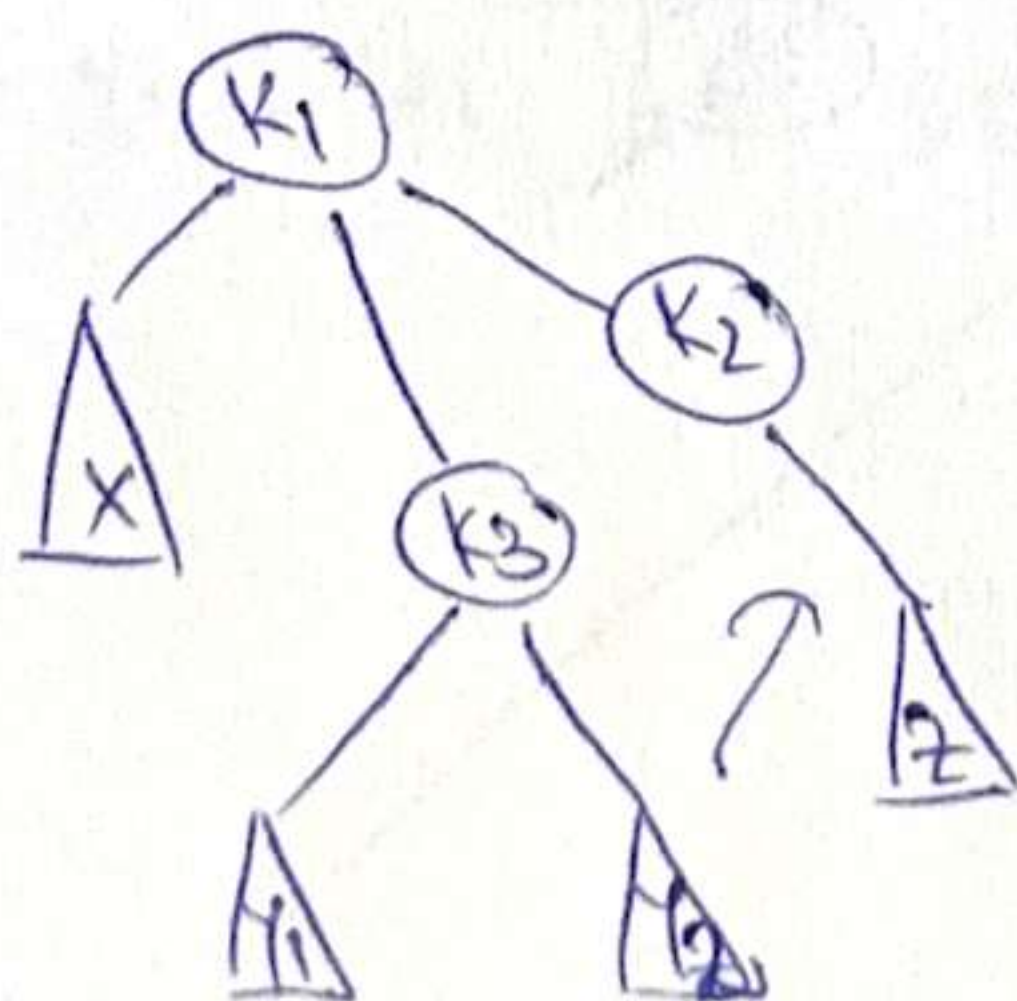
DRR

$\Rightarrow$

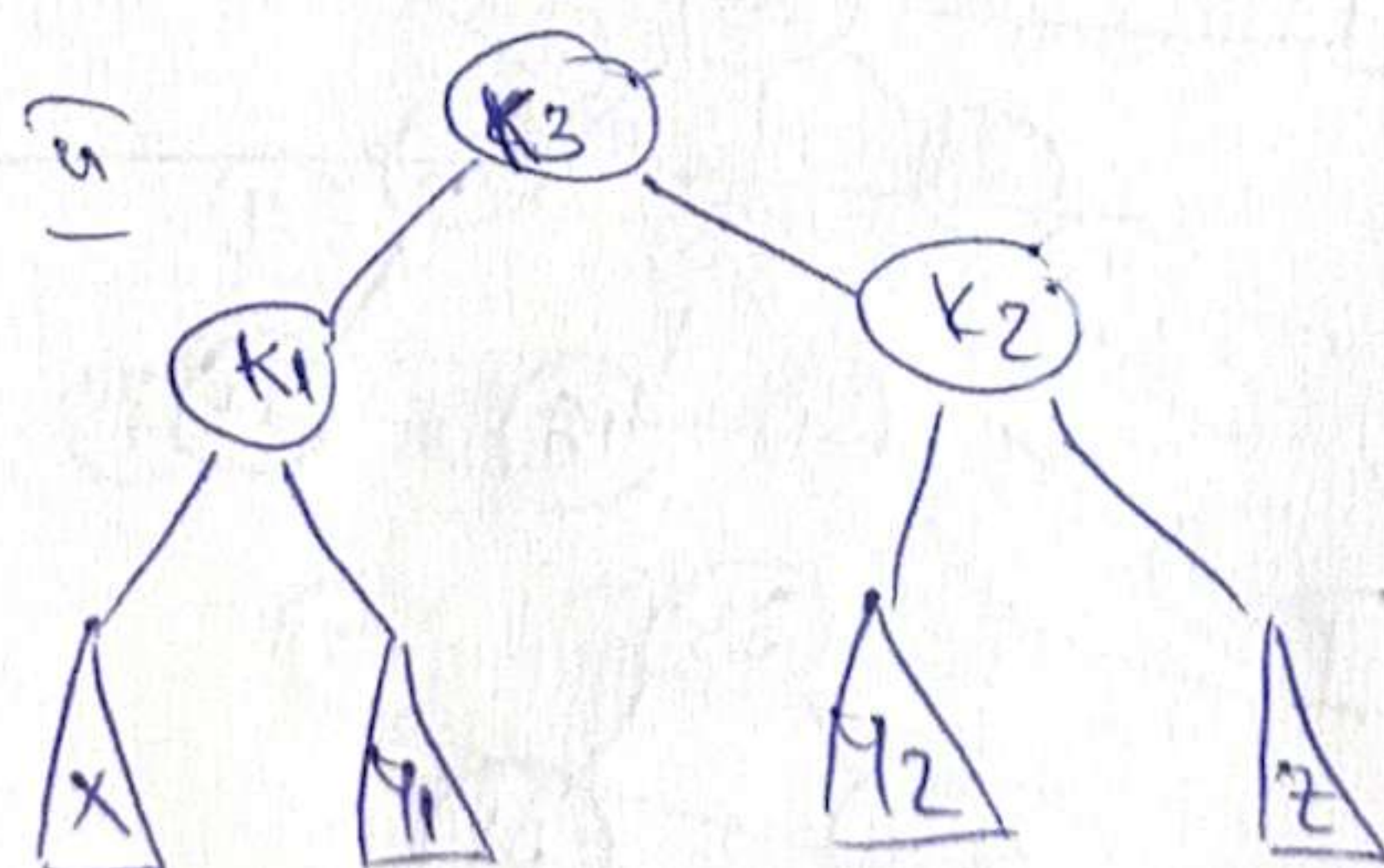
$\odot$



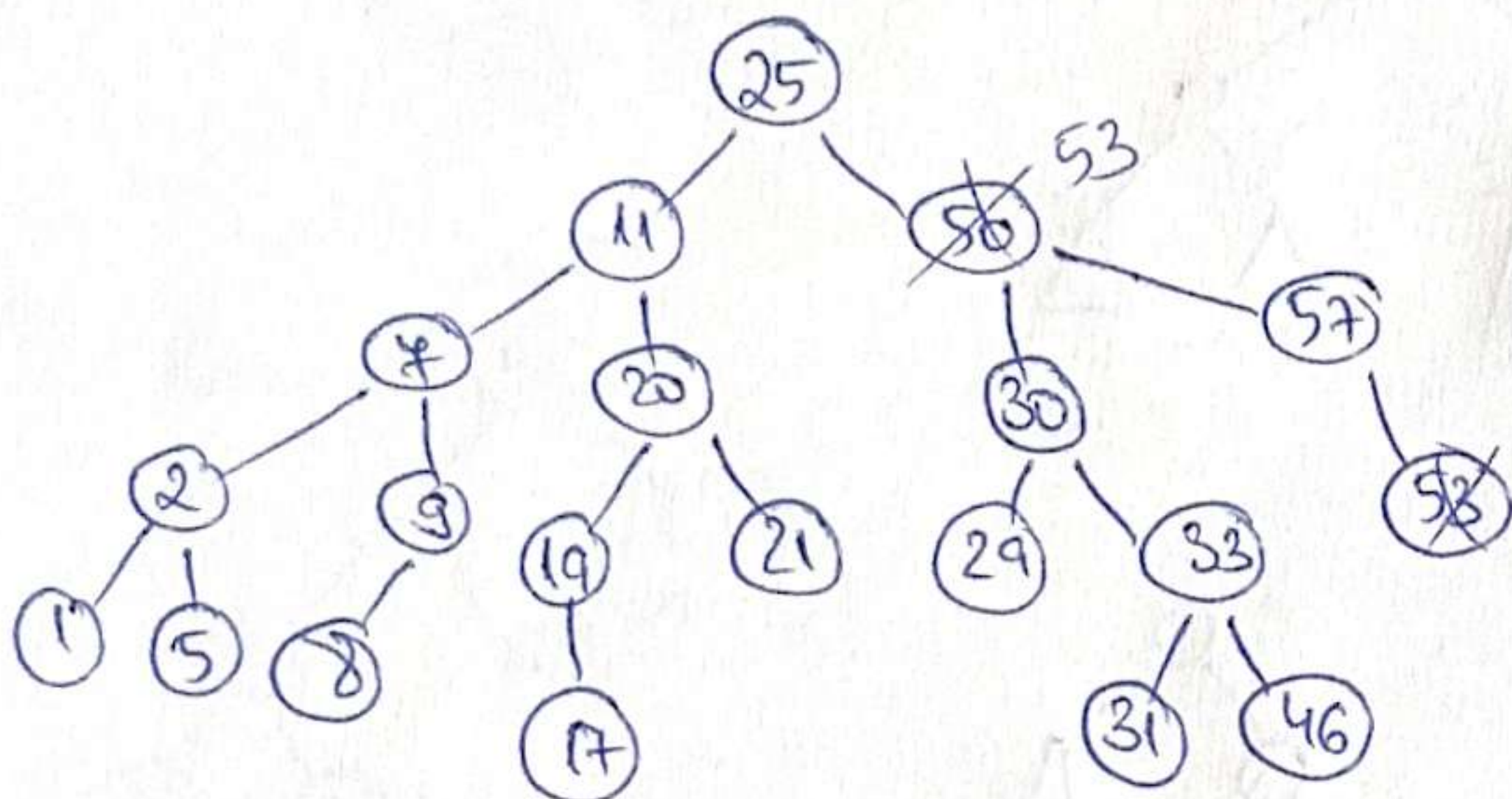
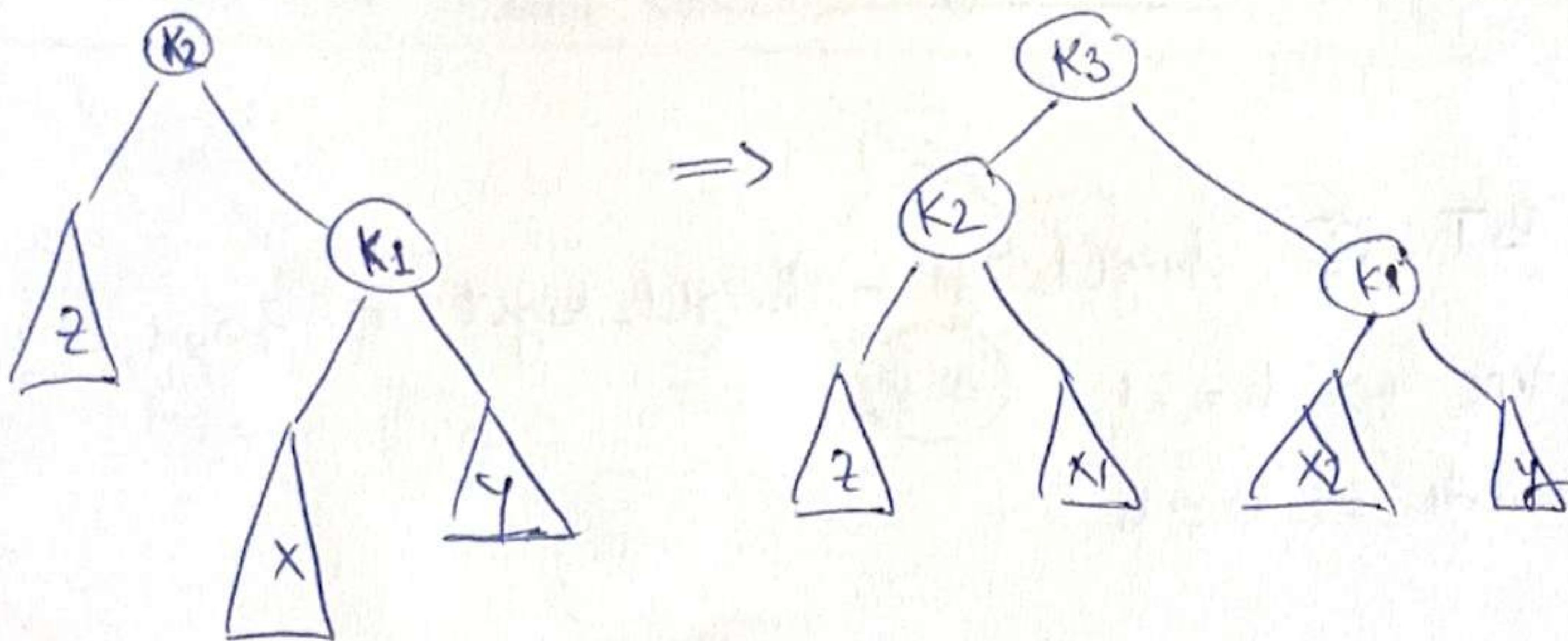
? $\odot$



(s)

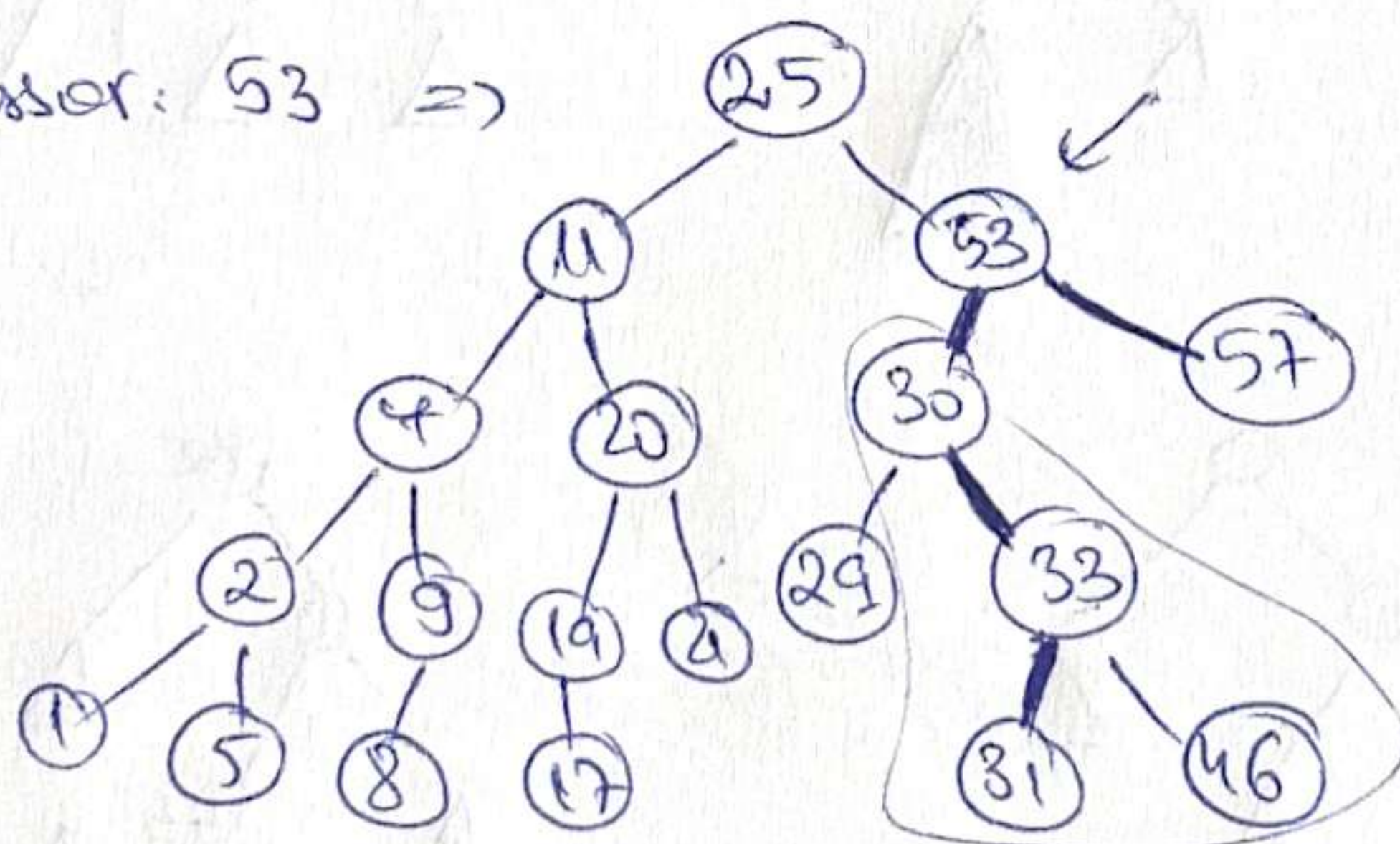


DRL:

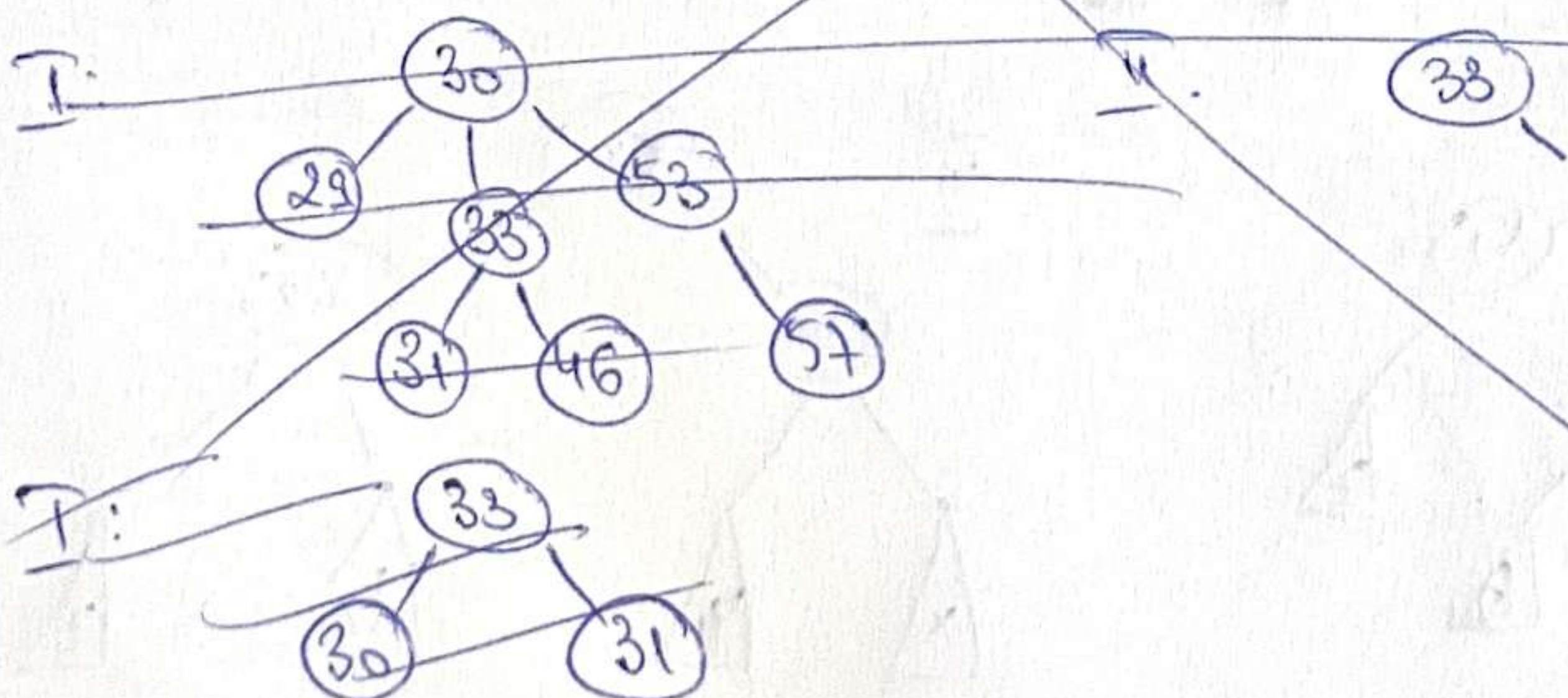
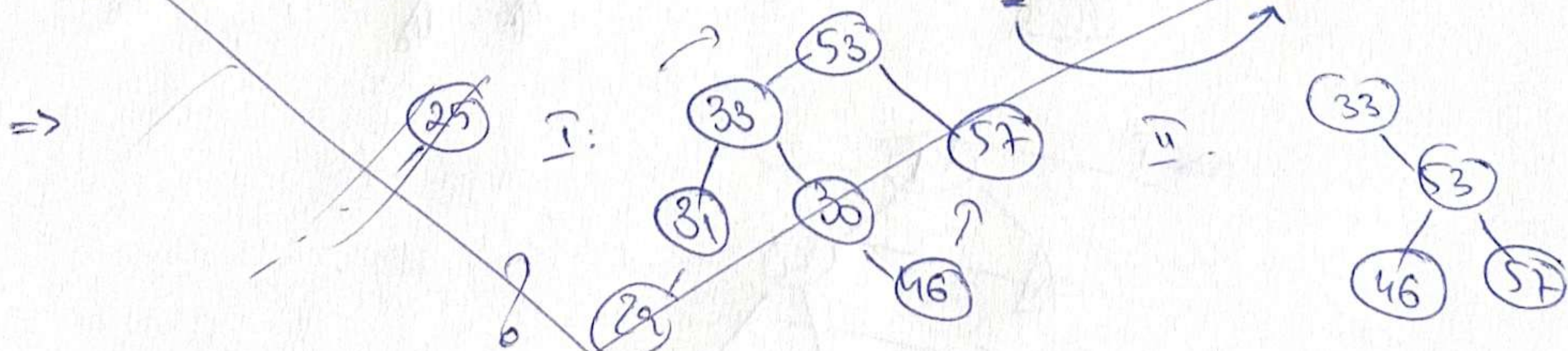


Remove 50

successor: 53 $\Rightarrow$



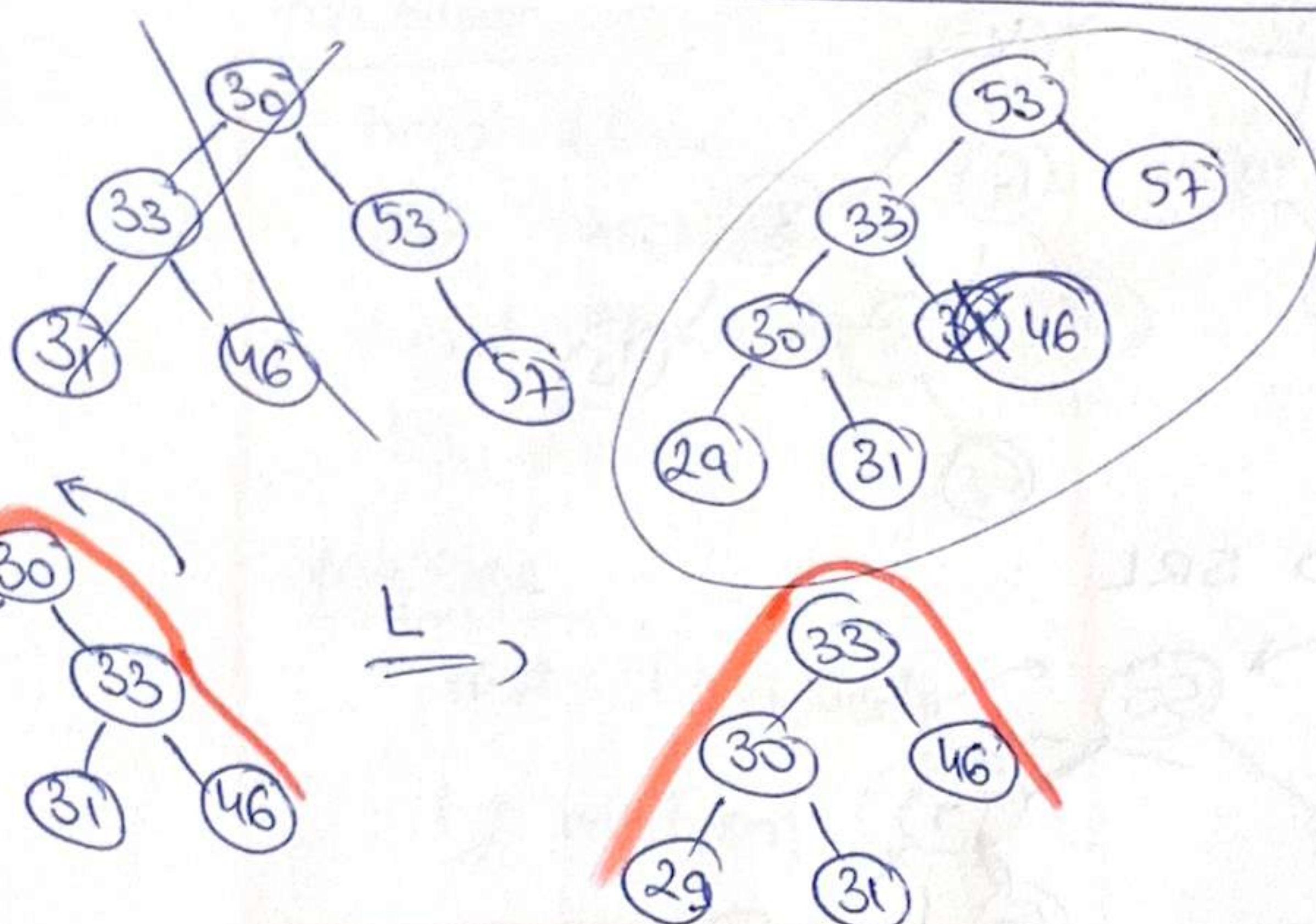
53: HL $\rightarrow$ its L child: 30 : HR $\Rightarrow$ LR $\Rightarrow$ DRR



DRR. does not mean rotate twice to the right!!!

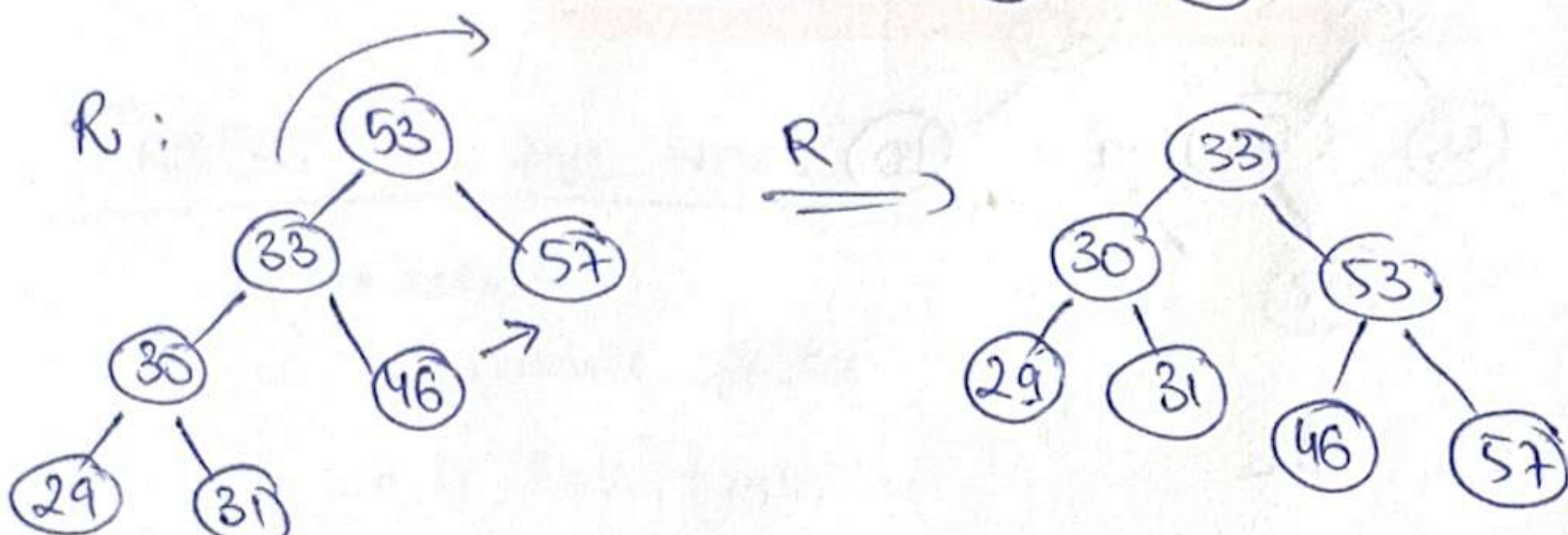
I & L:

L:

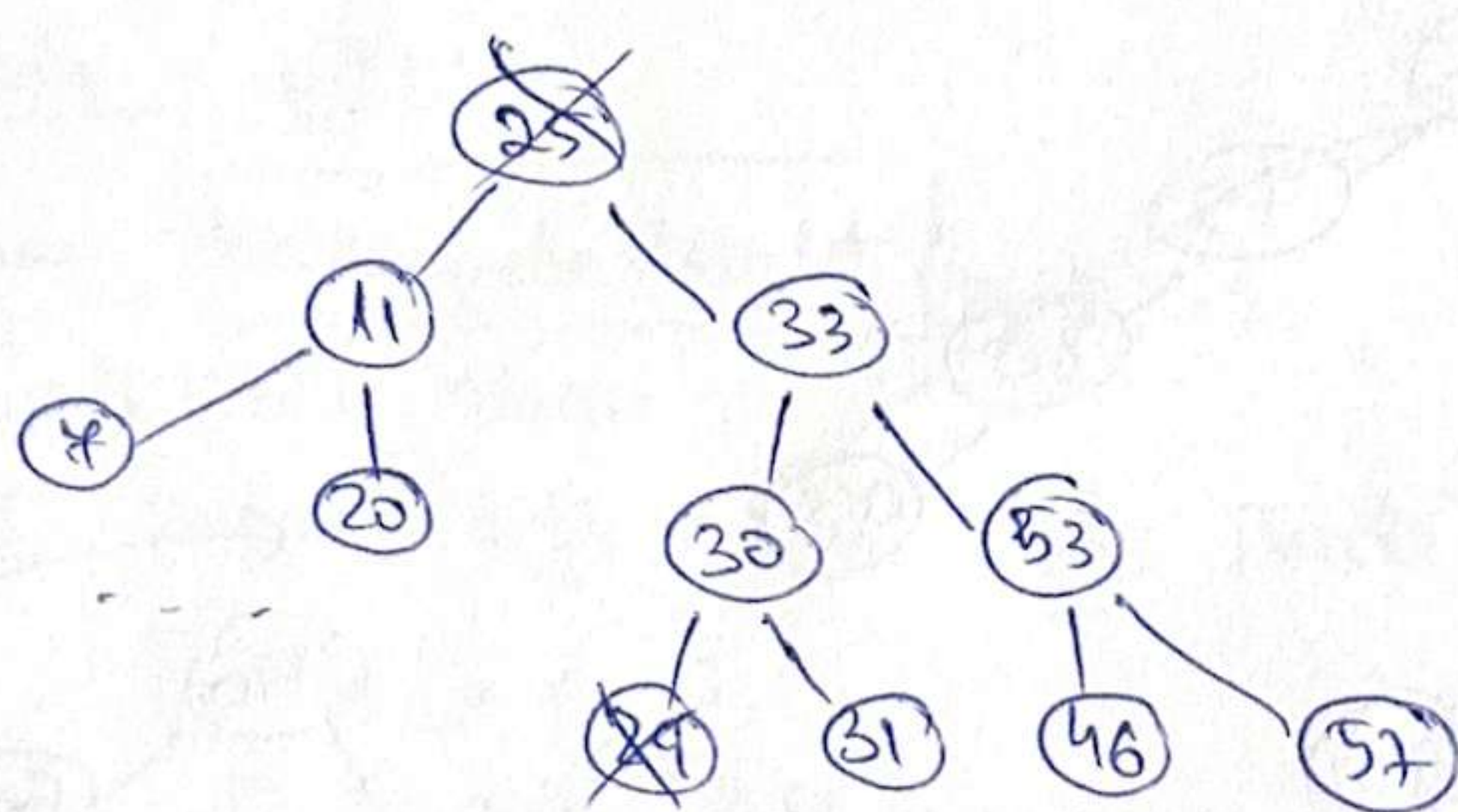


R:

R



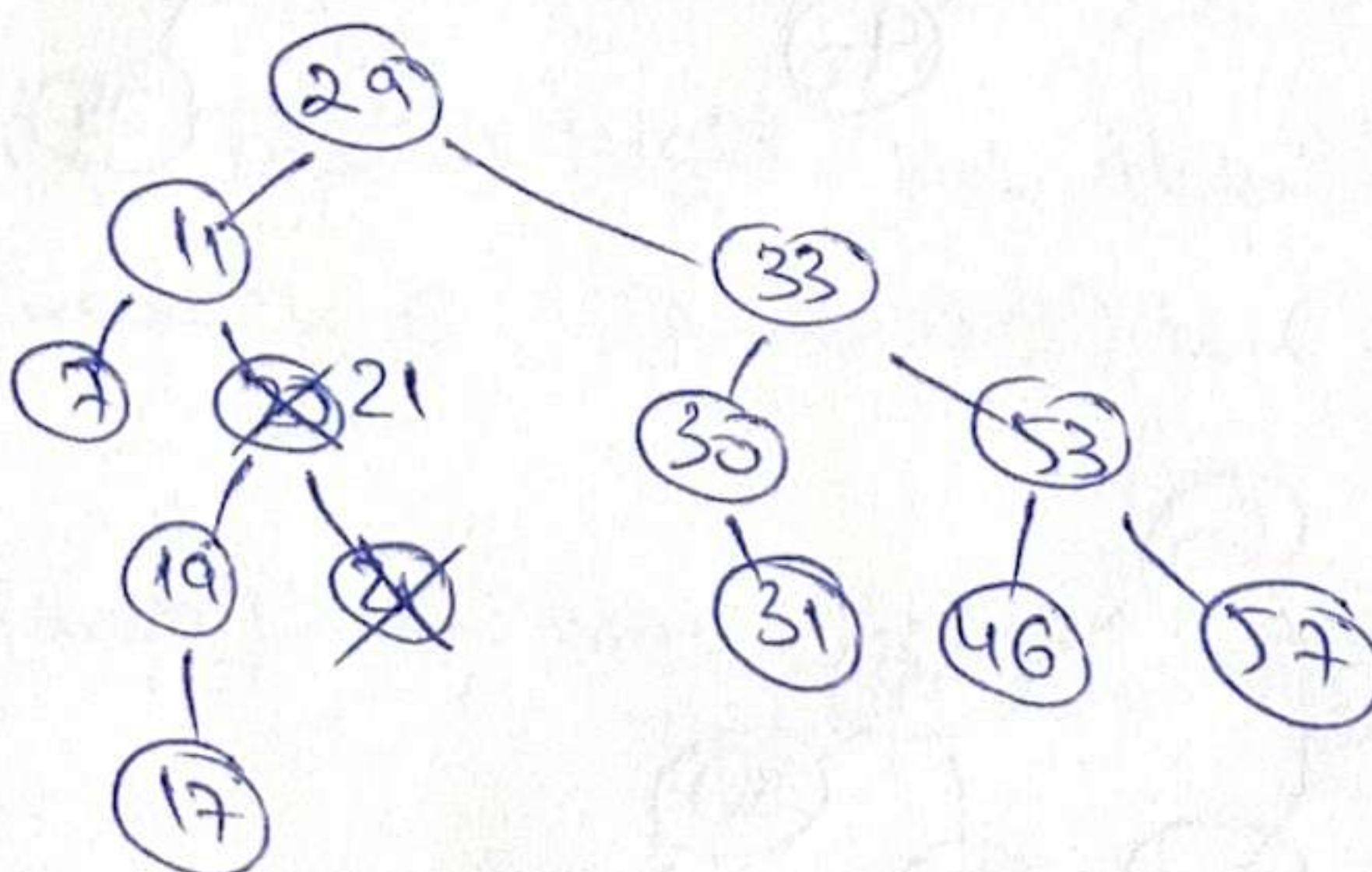
⇒



Remove 25

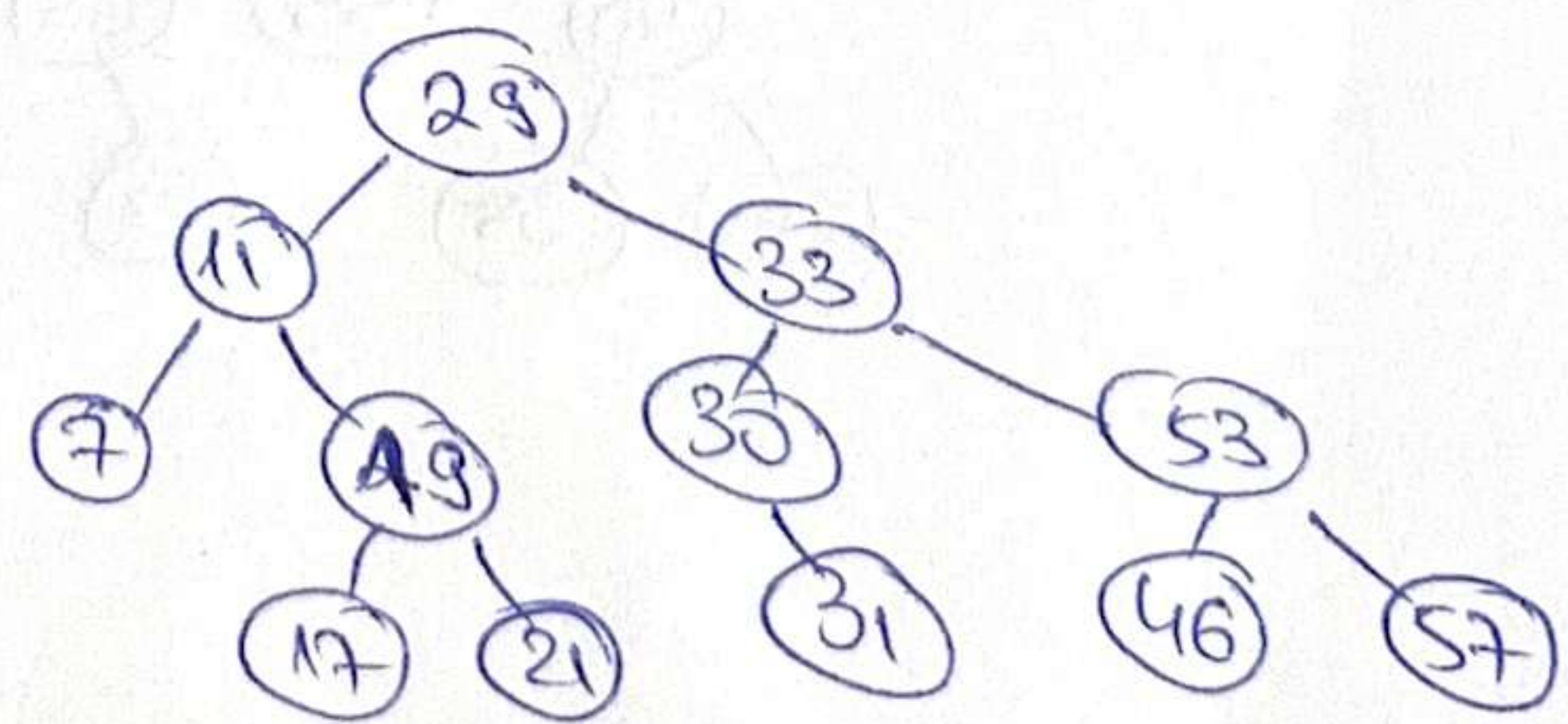
Successor: 29.

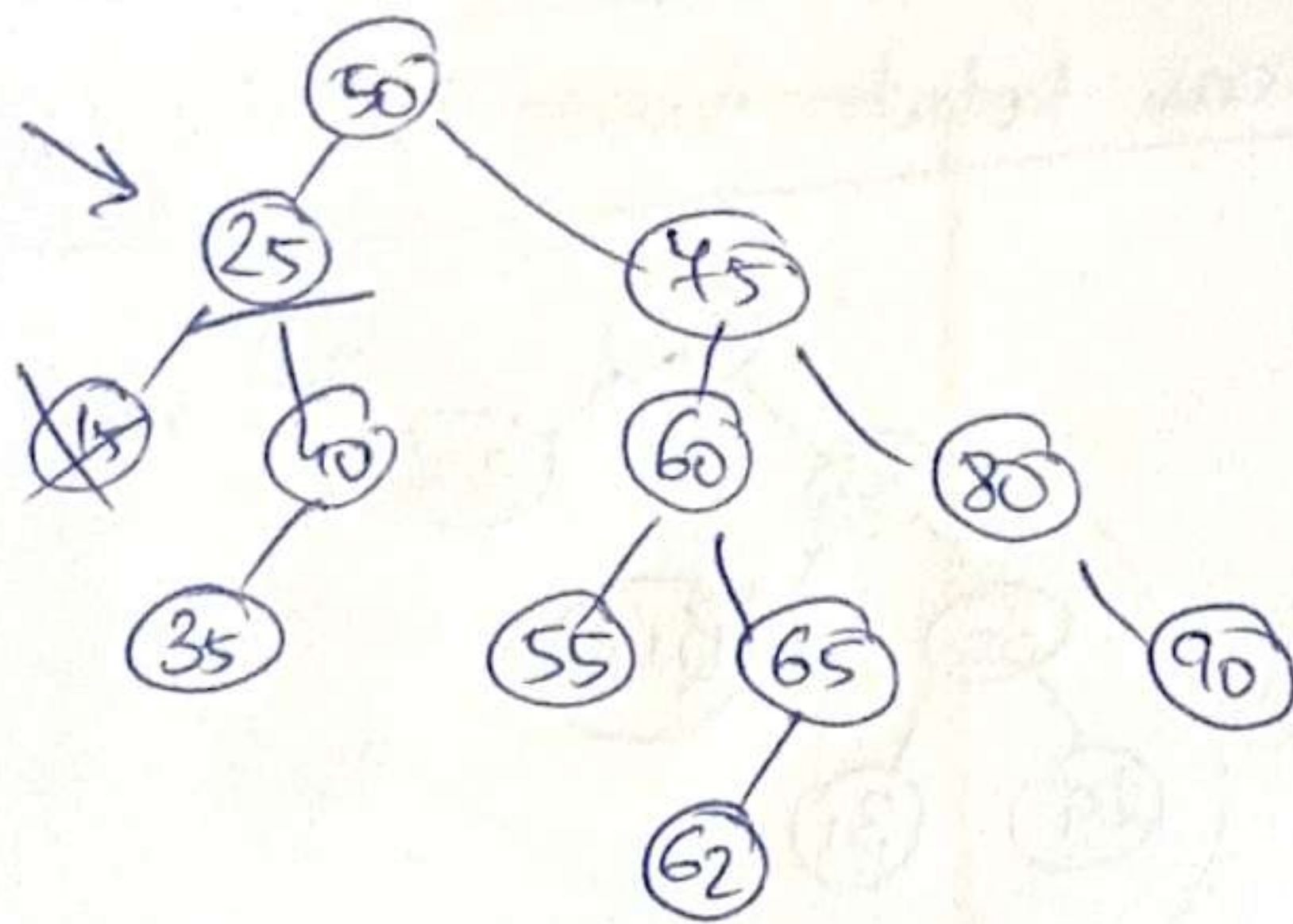
⇒



Remove 20

21: HL ⇒ SRR ⇒

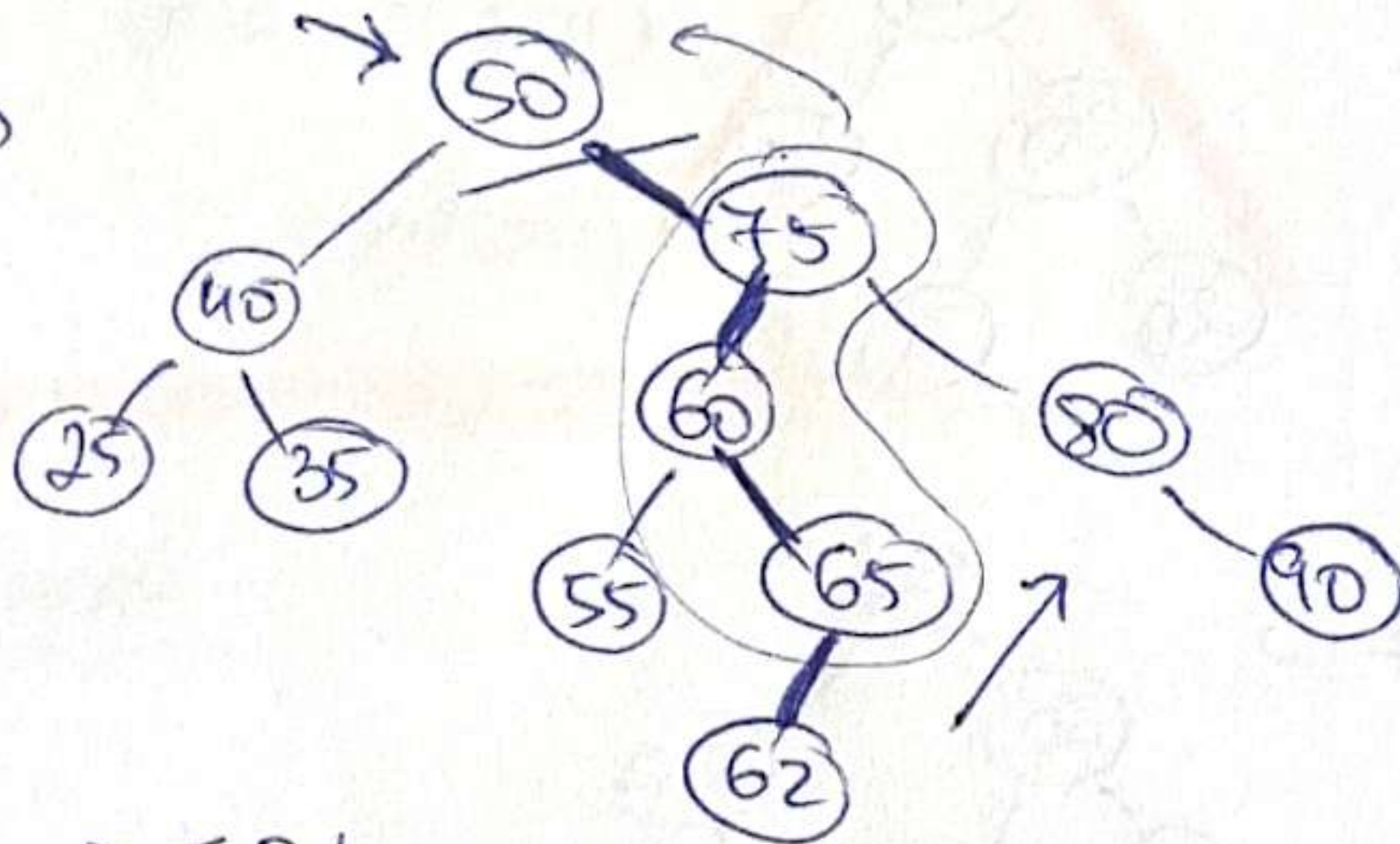




Remove 15

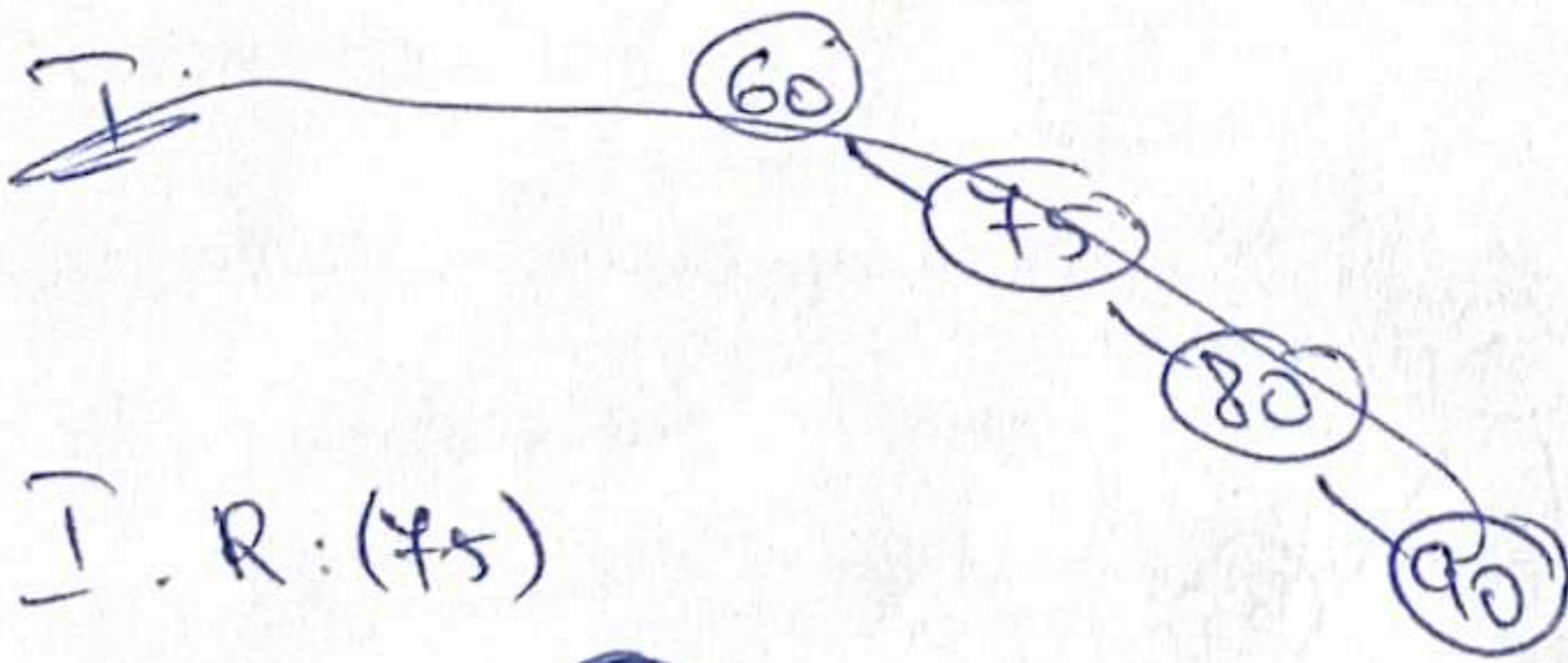
25: HR $\Rightarrow$ SRL

first imbalanced node from down to up



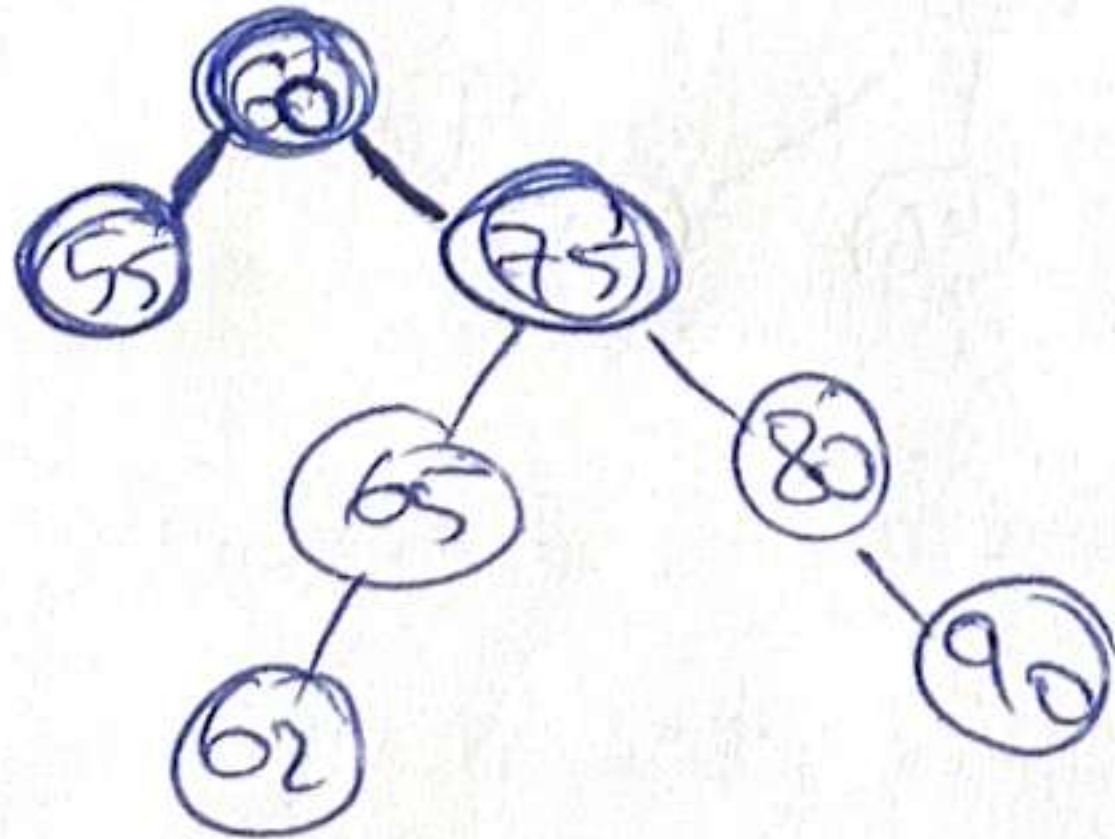
$\Rightarrow$ 50: HR $\Rightarrow$ ~~SRL~~

$\hookrightarrow$ 75: HL? $\Rightarrow$ RL

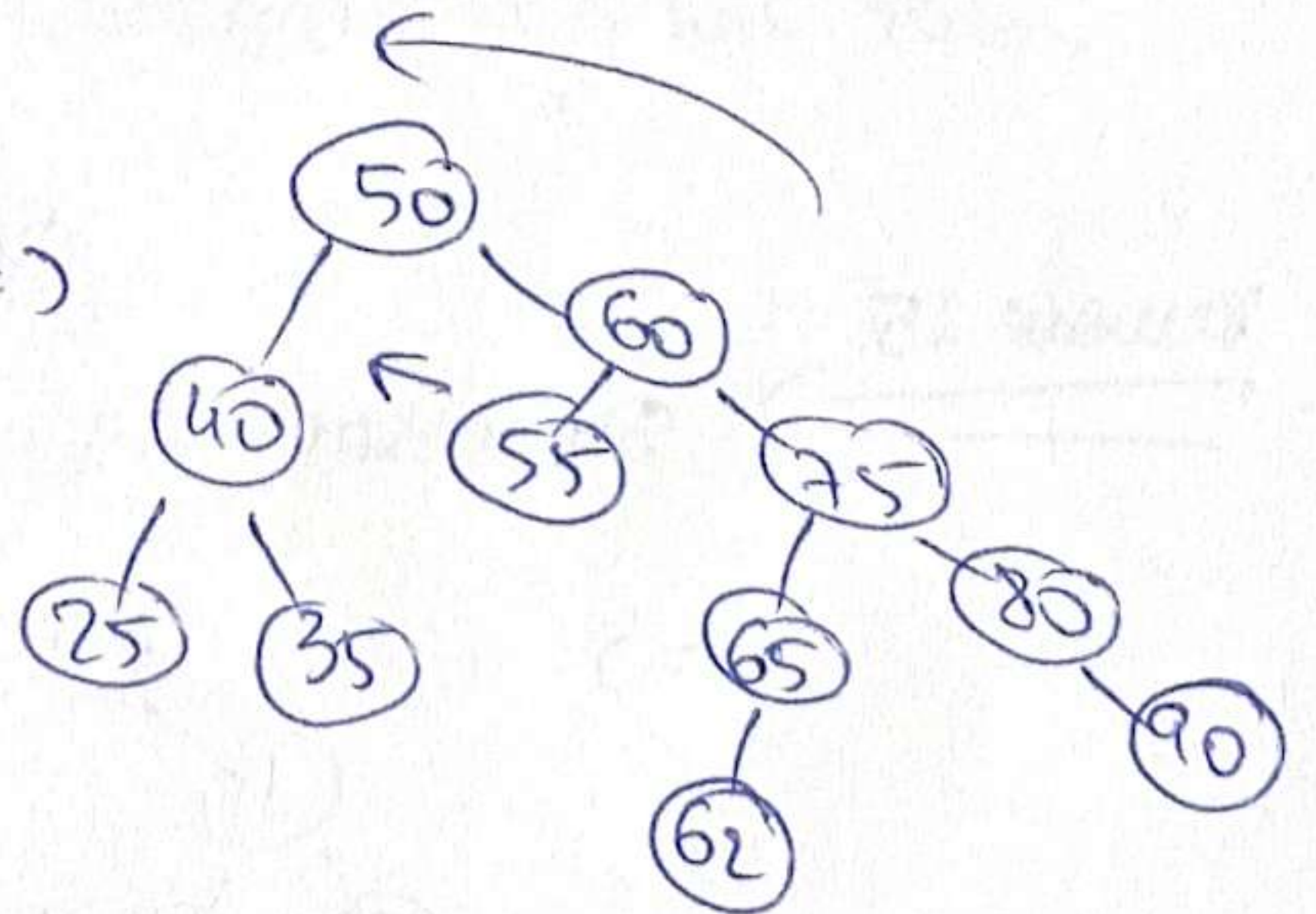


I. R: (75)

$\Rightarrow$

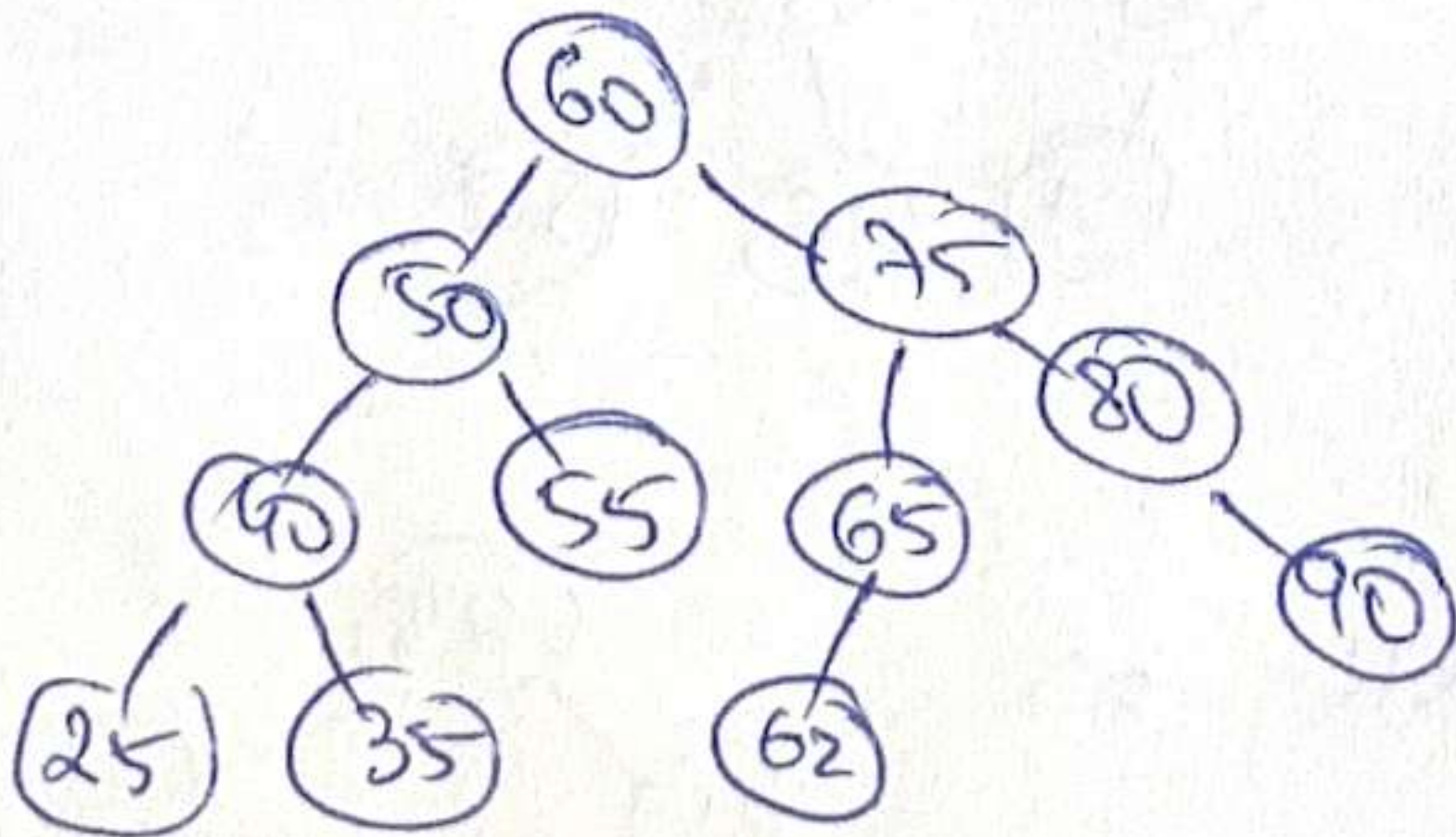


$\Rightarrow$



II: L: (50)

$\Rightarrow$



AVLNode

info: TElement
left: TAVLNode
right: TAVLNode
h: Int

AVLTree

root: TAVLNode
($\pm$ relation)

INSERT in AVL Node

- height
- balance factor
- 4 rot. types.

node: TAVLNode $O(1)$

subalgorithm recalculate Height (node) is:

if node $\neq$ Nil then

if [node].left = Nil and [node].right = Nil then

[node].h $\leftarrow 0$

else if [node].left = Nil then

[node].h $\leftarrow$ [node.right].h + 1

else if [node].right = Nil then

[node].h $\leftarrow$ [node.left].h + 1

else

[node].h $\leftarrow$ max ([node.left].h, [node.right].h) + 1

function balanceFactor (node) is:

if [node].left = Nil and [node].right = Nil then
balanceFactor $\leftarrow$ 0

else if [node].left = Nil then

balanceFactor $\leftarrow$ -1 - [node].right.h

else if [node].right = Nil then

function DRP (node) is:

K2 $\leftarrow$ node

K1 $\leftarrow$ [node].left

K3 $\leftarrow$ [K1].right

Insert recursively : $O(\log_2 n)$

$\hookrightarrow$ $\oplus$ wrapper!

$\downarrow$

Subalg insert(tree, elem) is

tree.root $\leftarrow$ insertRec (tree.root, elem)

Lecture 13

BTNode

info: TElem

left: $\uparrow$ BTNode

right: $\uparrow$ BTNode

BinaryTree

root: $\uparrow$ BTNode

~~Post~~

Root

$\uparrow$

Preorder

Left

Root

$\uparrow$

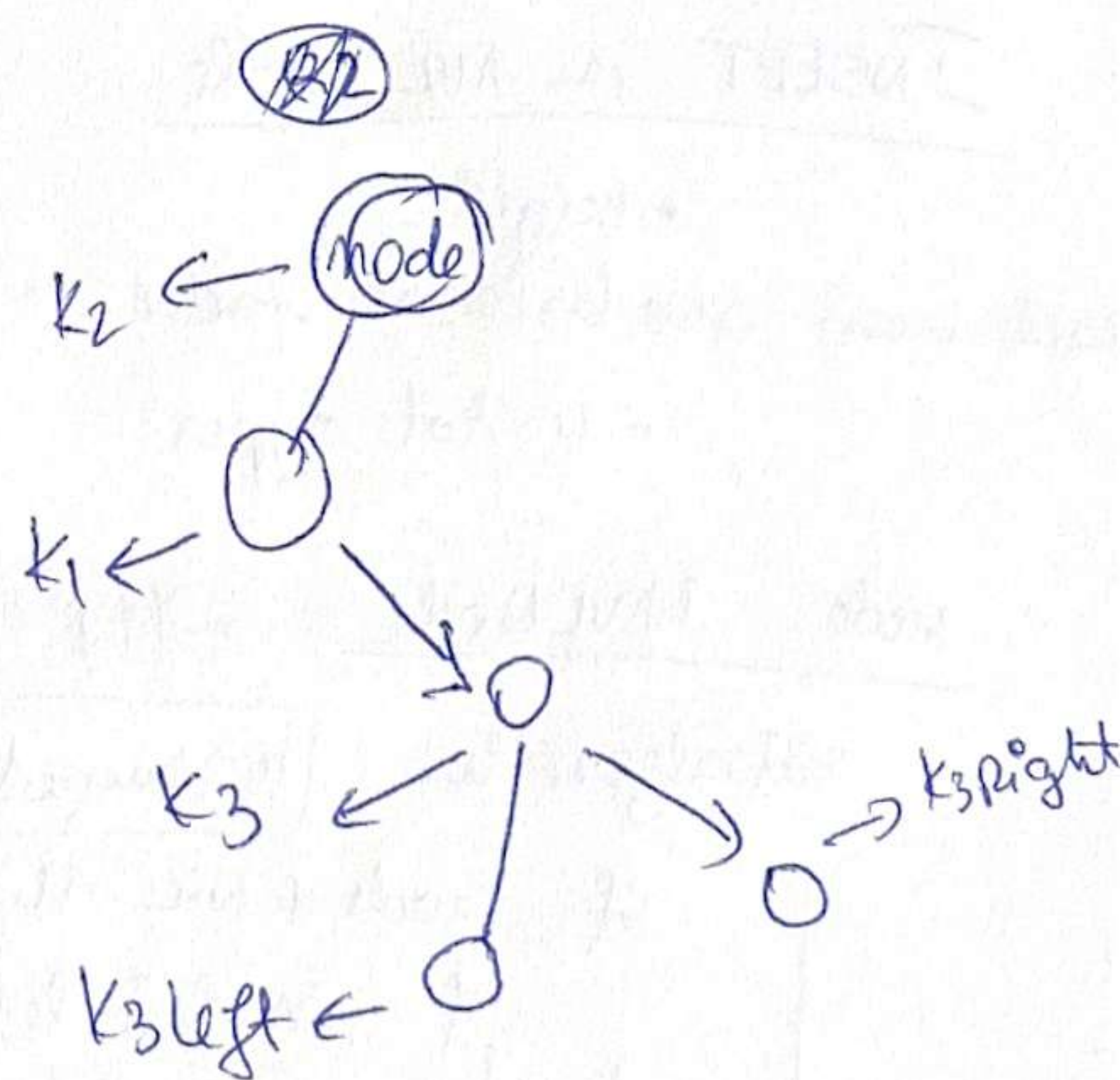
In order

Right

Root

$\uparrow$

Post order



postorder

subalg --- (node) is

if node $\neq$ Nil then

postorder_recursive([node].left)

postorder_recursive([node].right)

@ visit [node].info

Left
Right
Root

(M1)

(+) wrapper

$\Theta(n)$

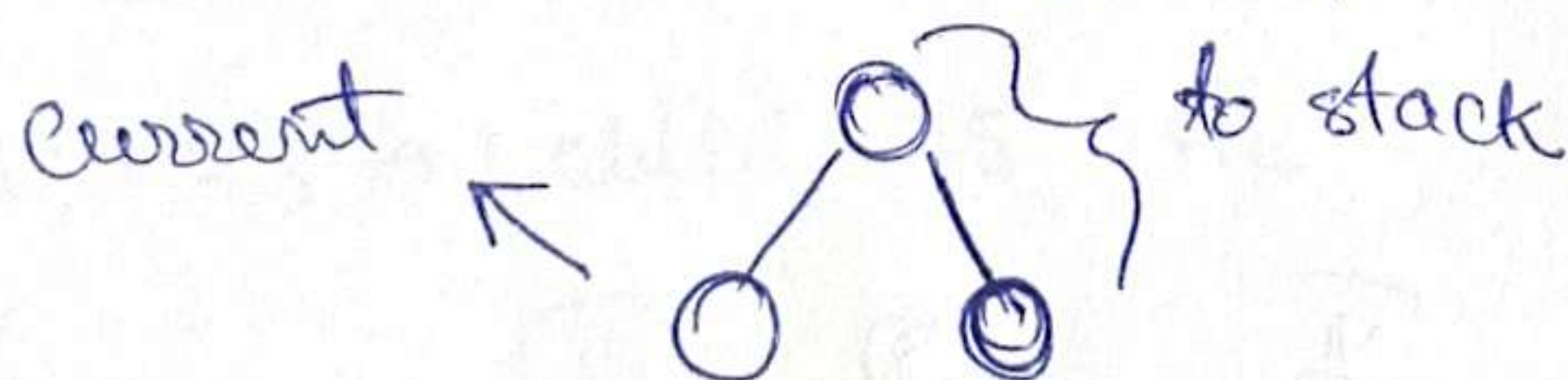
(M2) use two stacks

→ reverse of postorder in one stack → pop from stack

⇒ postorder

~ preorder but right subtree first

(M3) one stack ~~current~~



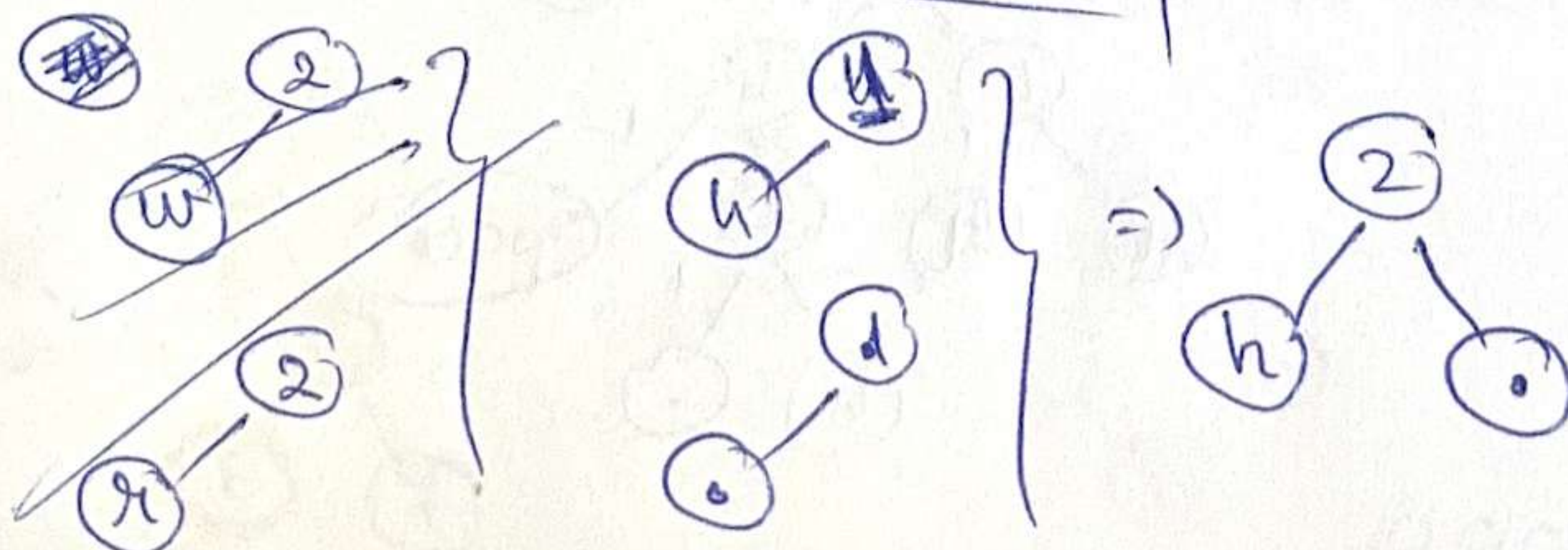
Huffman → Priority Queue.

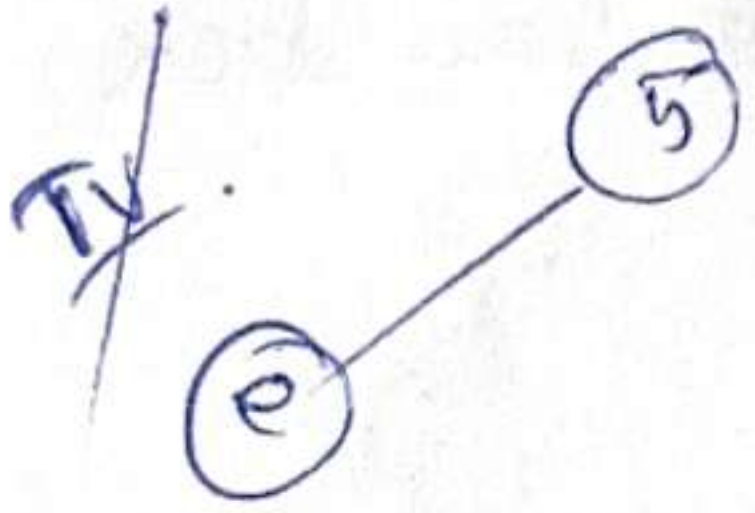
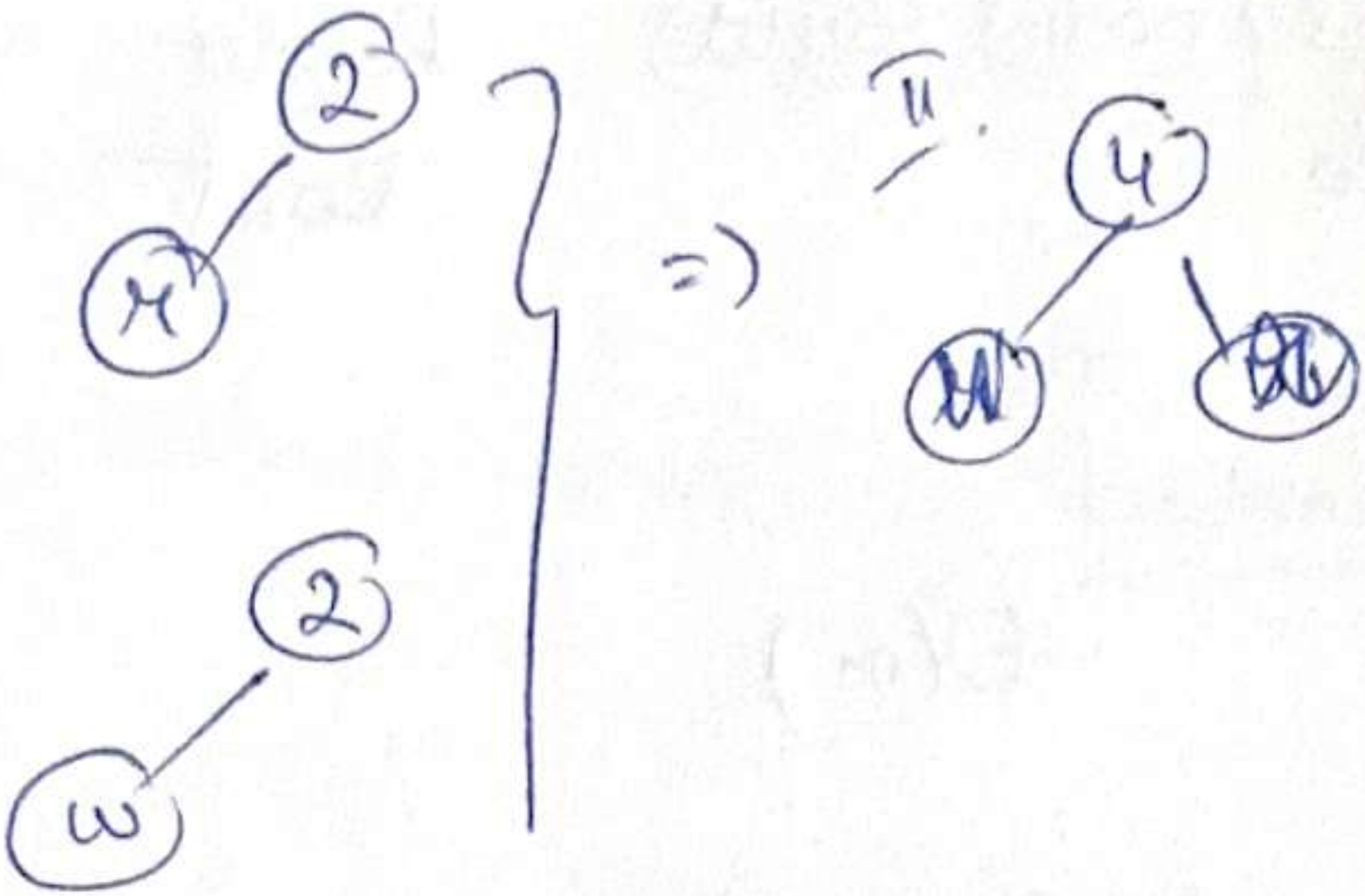
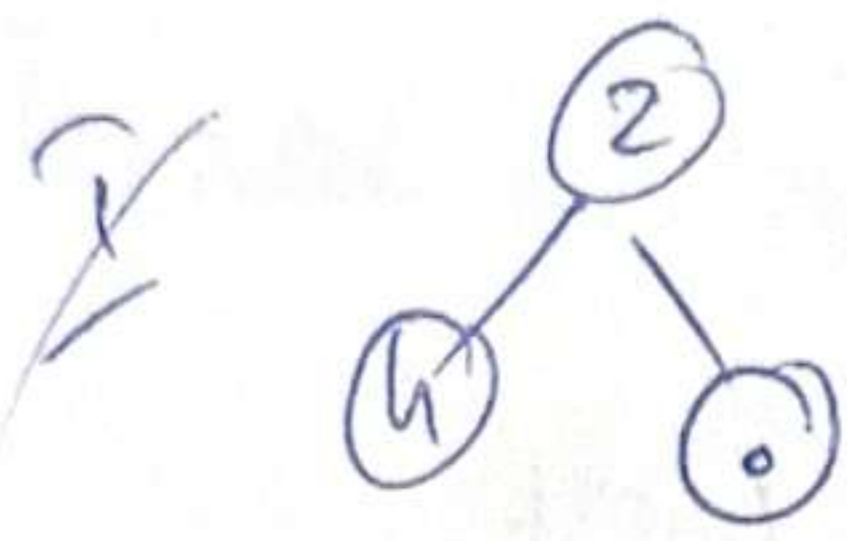
→ MESSAGE to encode.

→ FREQUENCY of every char in the MESSAGE

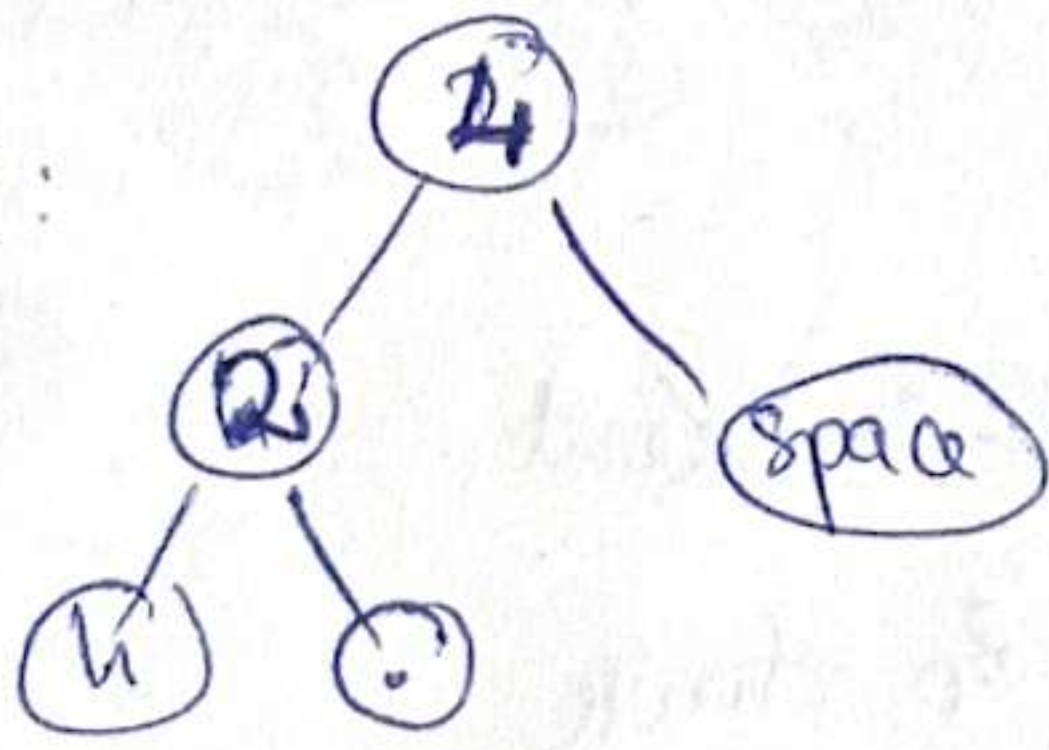
WE WERE HERE.

w	e	r	h	space	.
2	5	2	1	2	1

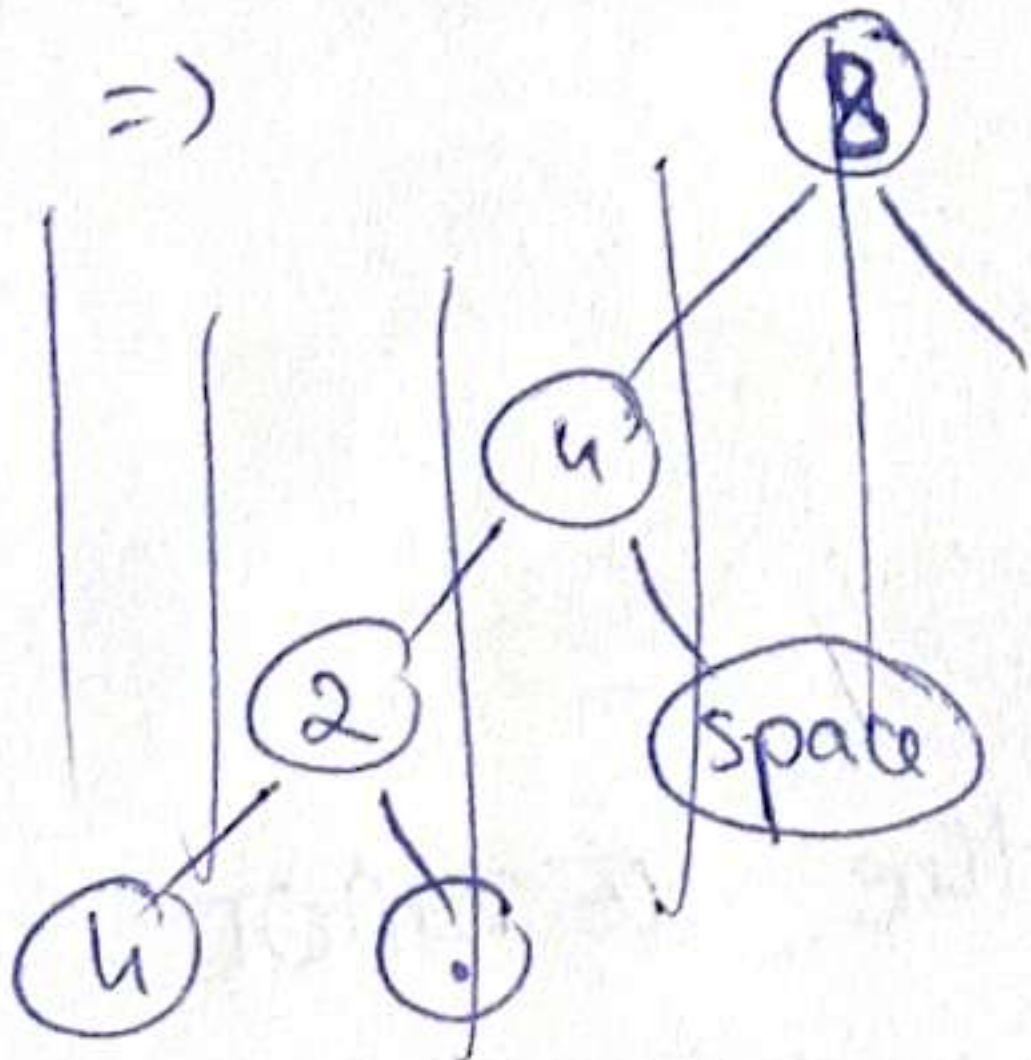




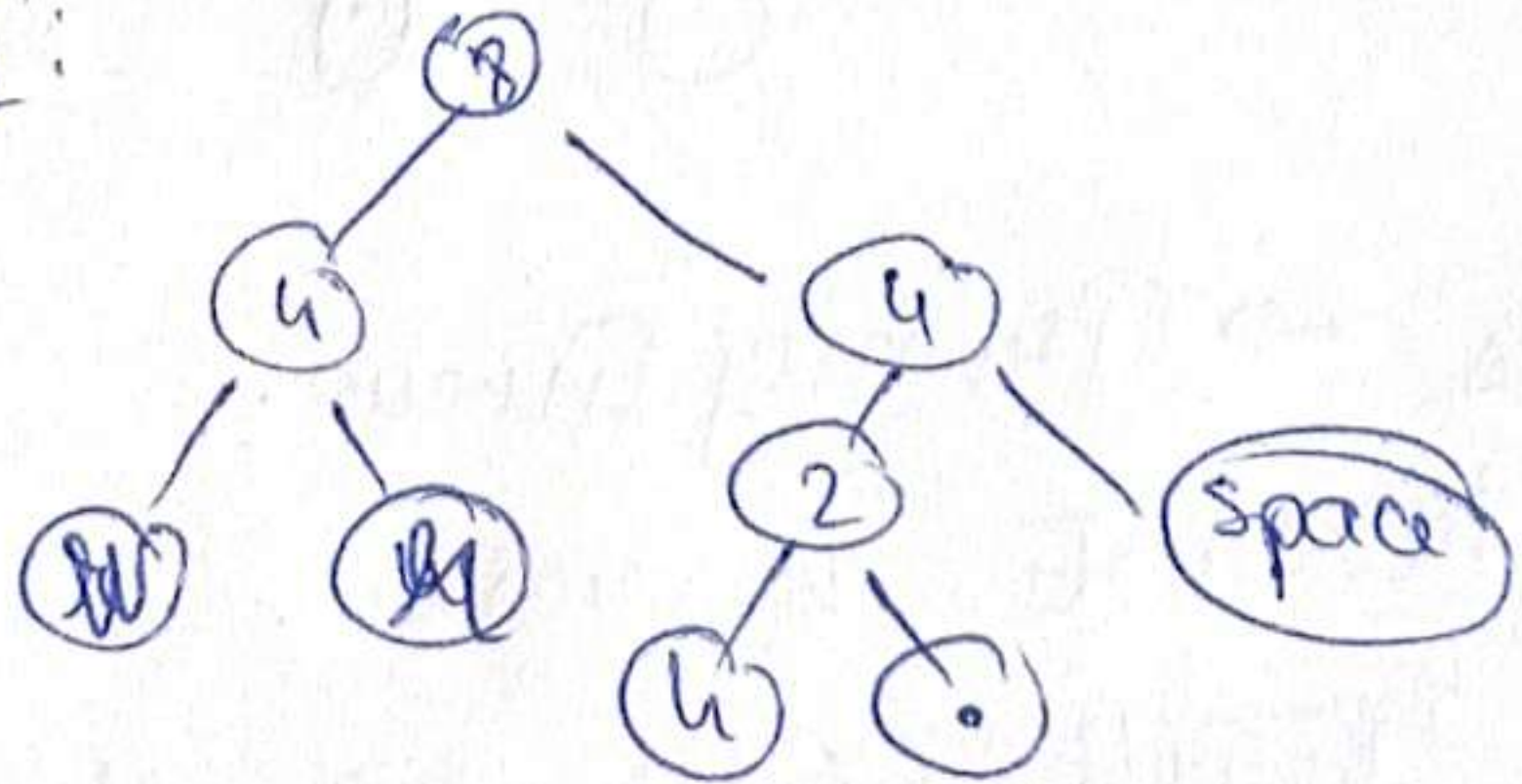
I, II $\Rightarrow$ V:



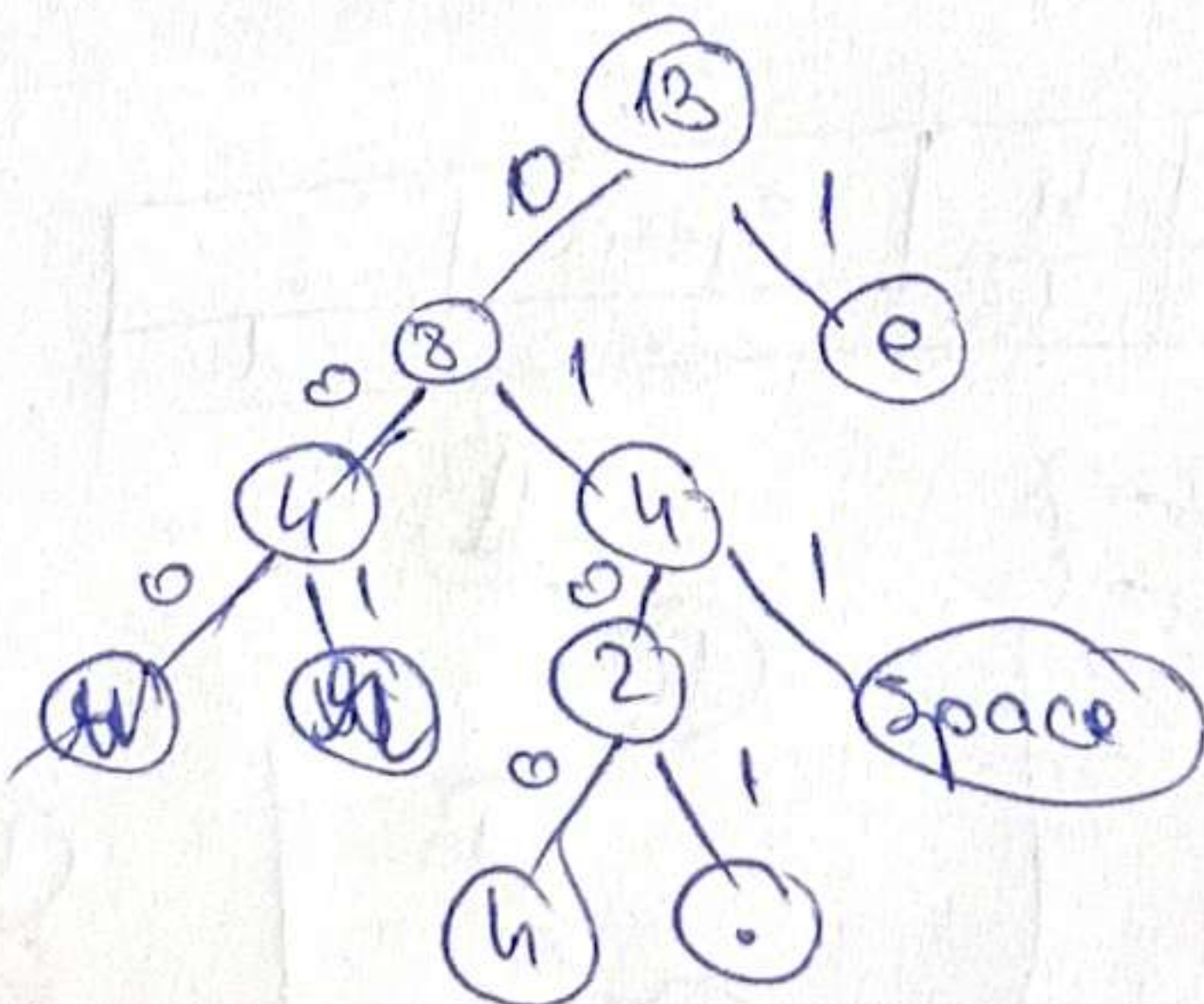
II, V $\Rightarrow$



VI:



VI + IV : $\Rightarrow$



$w \Rightarrow 000$

$\pi \Rightarrow 001 \dots$

Inorder on BST $\Rightarrow$ increasing order.

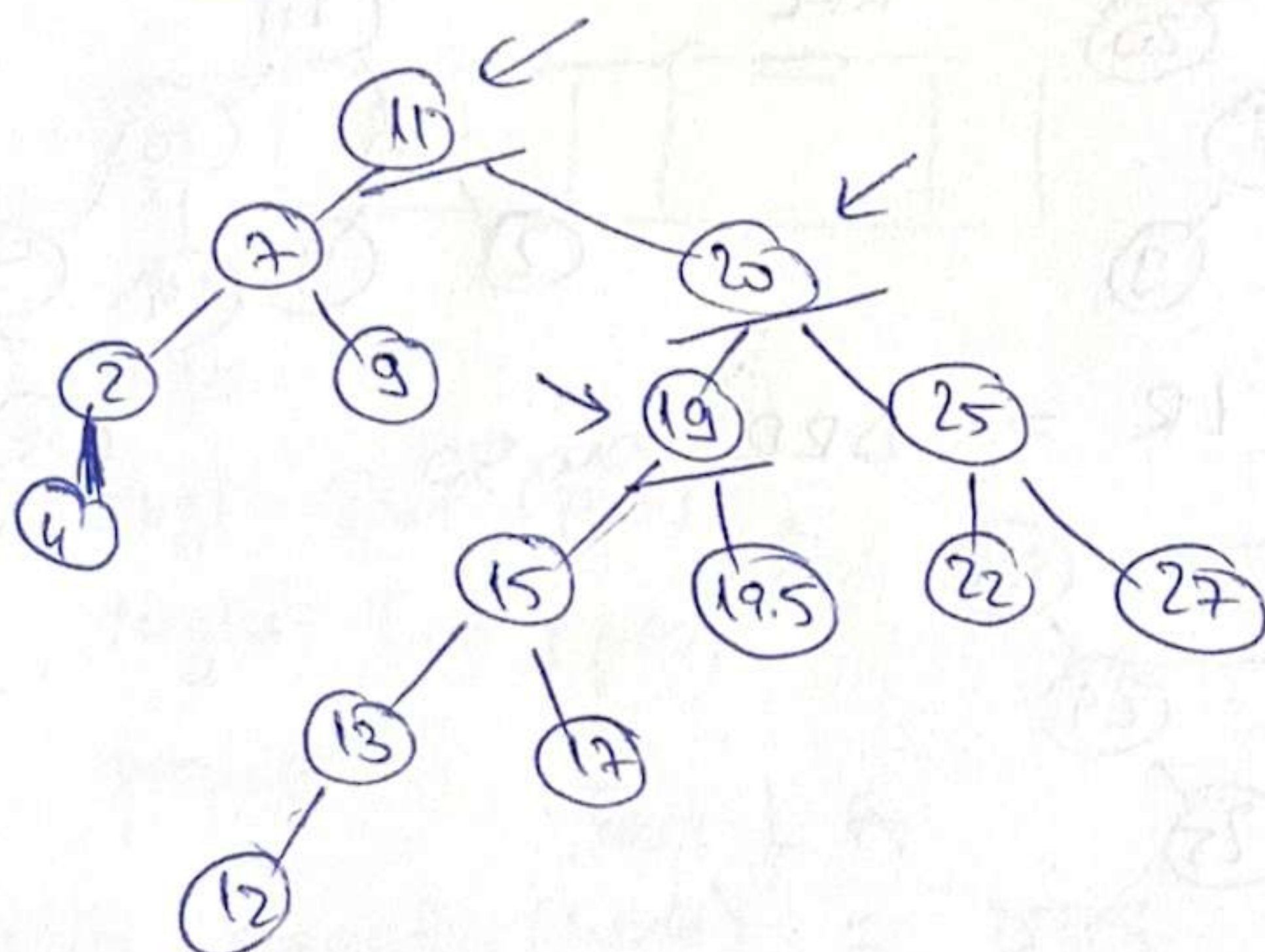
$O(h)$, $\Theta(m)$ WC.

$\Theta(\log_2 m)$ BC. (balanced)

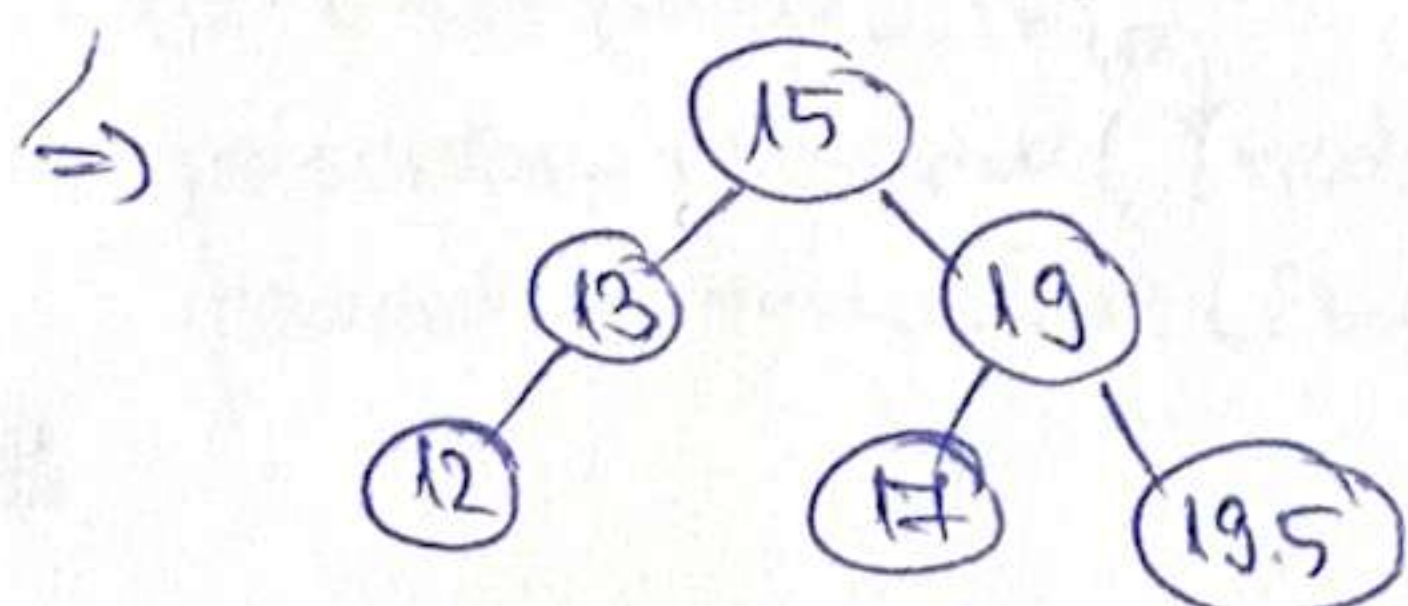
$\underbrace{\hspace{1cm}}_h$

reduce compl $\Rightarrow$ balanced. $\Rightarrow$ AVL.

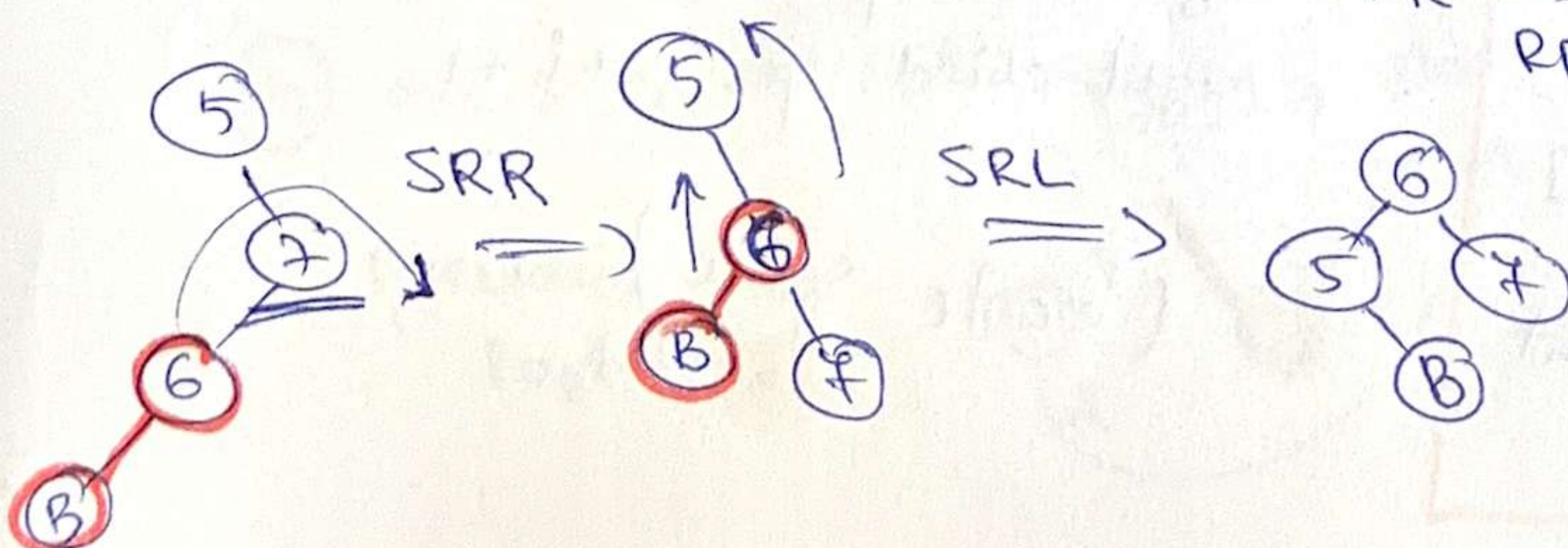
$\uparrow$ check balance.



19: HL, $\rightarrow$ Lchild: 15 : HL $\Rightarrow$ LL $\Rightarrow$ SRR. on 19.



imbalanced $\rightarrow$ HL $\rightarrow$ left child $\rightarrow$ HL $\Rightarrow$ SRR (LL)
 $\rightarrow$ HR $\Rightarrow$ DRR (LR)
 $\rightarrow$ HR $\rightarrow$ right child $\rightarrow$ HL $\Rightarrow$ DRL (RL)
 $\rightarrow$ HR $\Rightarrow$ SRL (RR)



Insert in new AVL tree (empty)

=> (2) =>

(2)

(11) =>

(2)
(11)

(20) =>

(2)
(11)
(20)

SRL

(10)
(2) (20)

(8) =>

(11)
(2) (20)
(8) (9)

RRL

(11)
(8) (20)
(2) (9) (19) (50)
(25) (29)

(50) HL => 25 HR => LR => DRR on 50.

T: L:

(29) (50)
(25) (29)
(25)

u: R:

(29)
(25) (50)

Lecture 12

ADT Binary Tree

I. on DLLA

Node:

info: TElem

left: Int

right: Int

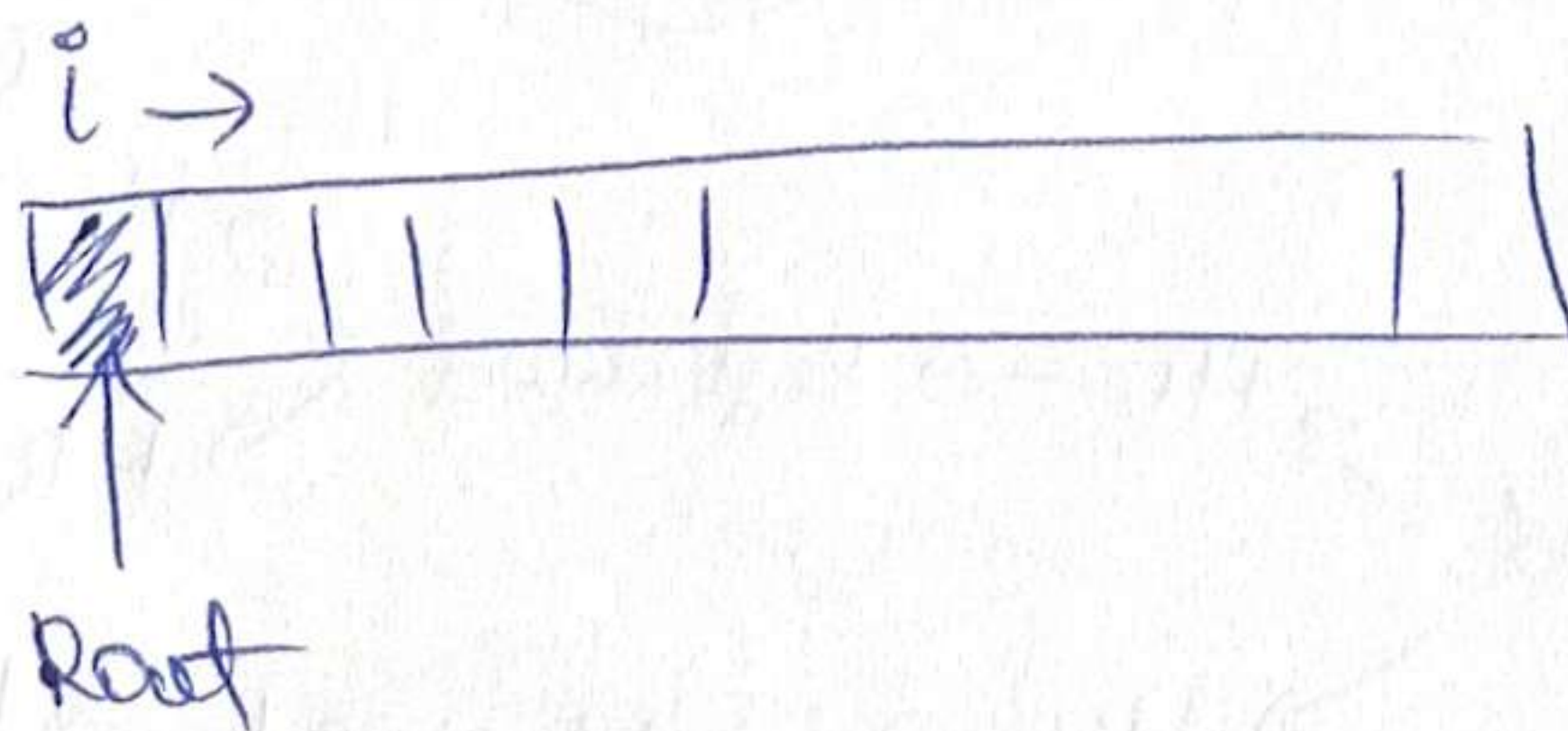
BT:

nodes: TNode[]

root: Int

firstEmpty: Int

exp: Int



left child: pos $2*i$

right child: pos $2*i + 1$

(waste space)

II. LL w Dyn. alloc

Node

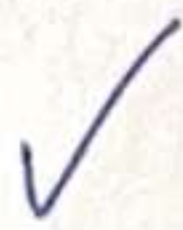
info: TElem

left: $\uparrow$ Node

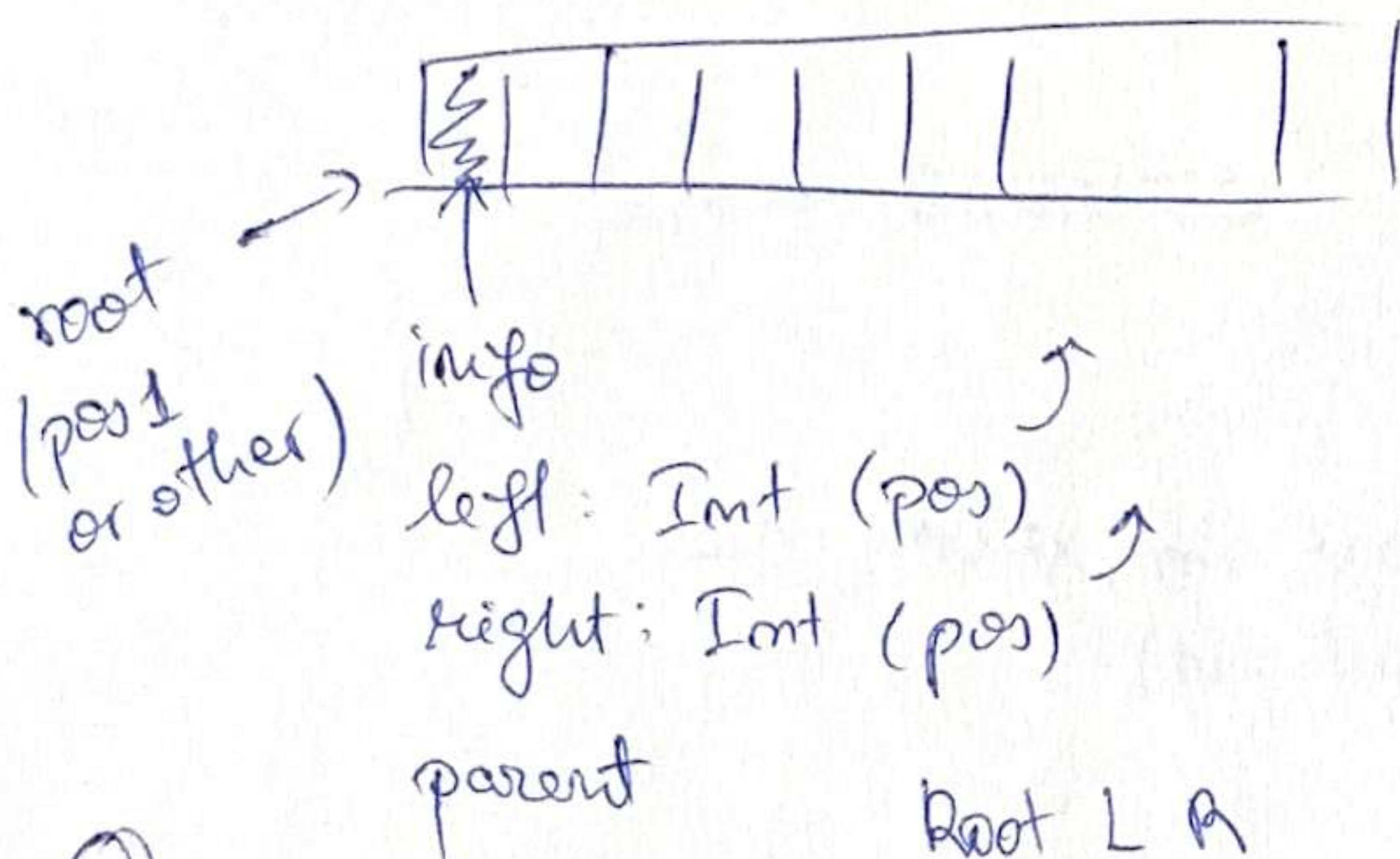
right: $\uparrow$ Node

BT:

root: $\uparrow$ Node



III. LL on array (LL of empty pos.)



pos	
info	
L	
R	
parent	

(M1) subalg preorder(node) = DFS but R then L. to stack.

if node $\neq$ Nil then

@ visit [node].info

preorder-recursive([node].left)

preorder-recursive([node].right)

inorder

postorder

(+) Wrapper!

subalg preorderR(tree) is:

preorder-recursive(tree.root)

$O(n)$

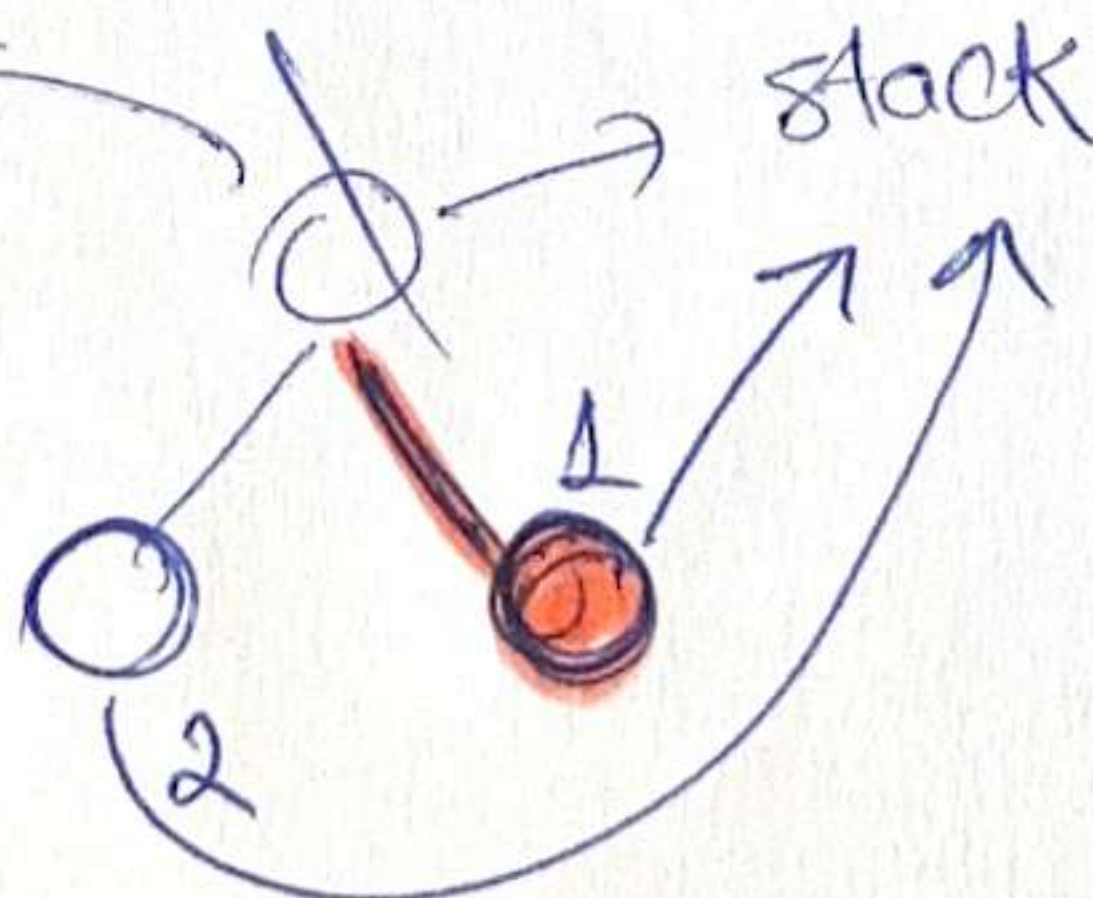
(M2) aux stack

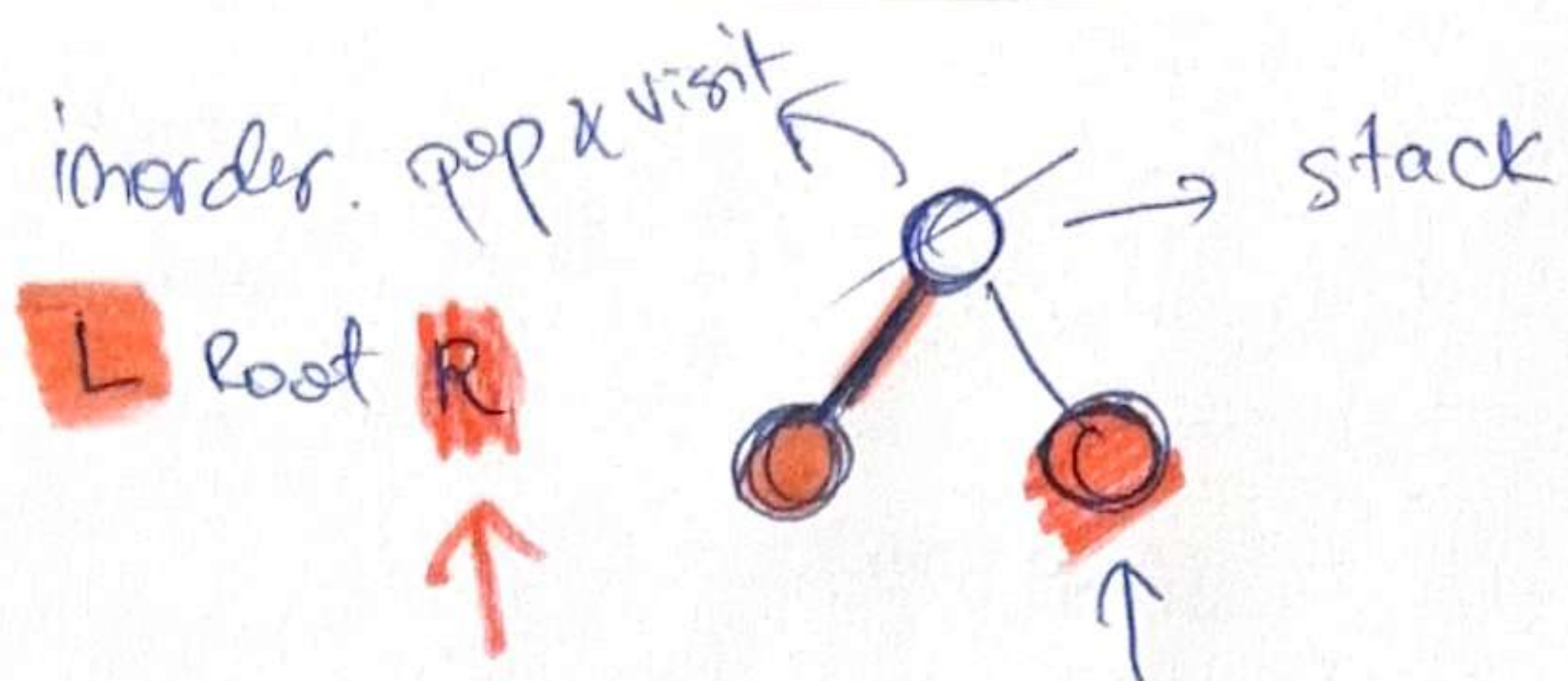
pop & visit

stack

preorder

Root L **R**





Inorder / Preorder w stacks again!

BST.

→ FULL

→ COMPLETE

→ ALMOST COMPLETE

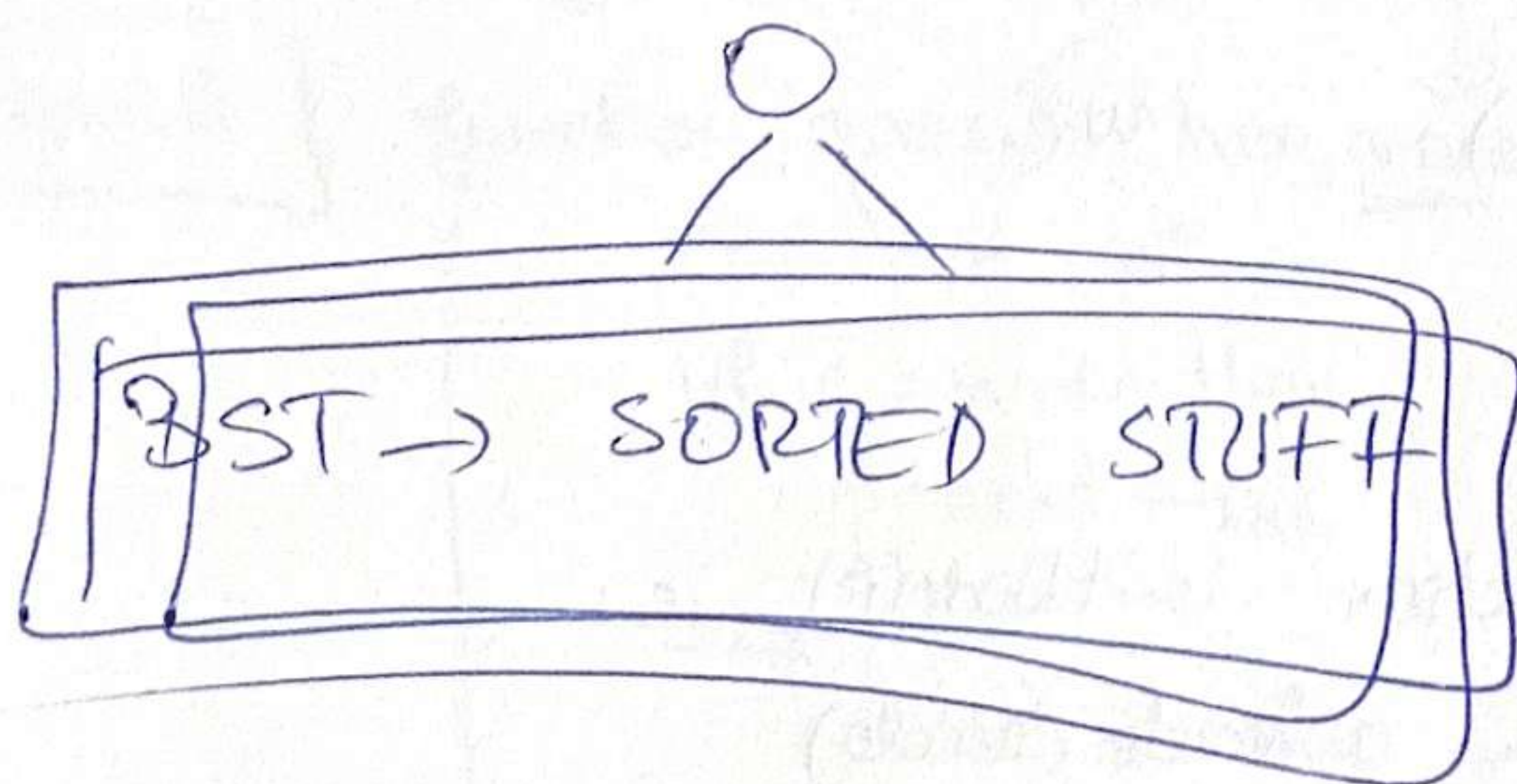
→ DEGENERATE

→ BALANCED

→ max. no. nodes in a BST : $2^{N+1} - 1$
(N = height) (complete)

→ min BST : N
(degenerate)

→ $h \in (\log_2 N, N)$



min / max

Search

insert

remove

Successor / predecessor.

Search

function search(node, elem) is

if node = Nil then
search ← false] BASE CASE!

else

if [node].info = elem then

search ← true

else if [node].info < elem then

search ← search([node].right, elem)

else

search ← search([node].left, elem)

⊕ wrapper:

function searchwrapper (tree, elem) is:

~~search ← sea~~

searchwrapper ← search (~~sea~~ tree.root, elem)

Worst BST let → one path (root-to-leaf) ⇒ $\Theta(h)$.
 $h \in (\log_2 n, n)$ } ⇒

⇒ $\Theta(\log_2 n), \Theta(n) \rightarrow \sim O(n)$.

↓
Bc
 $\Theta(1)$

↓
Ac
(search)

↓
Wc

Insert

allocate → function initNode(e) is:

allocate (node)

[node].info ← e

[node].left ← Nil

[node].right ← Nil

initNode ← node

function insert (node, e) is:

```

if node = Nil then
    node ← initNode (e)
else if [node].info ≥ e then
    [node].left ← insert ([node].left, e)
else
    [node].right ← insert ([node].right, e)
insert ← node

```

Ac: $\Theta(\log_2 m)$
 Wc: $\Theta(m)$ } $\Rightarrow O(m)$

OR node (for subtree)!

min

function minimum (tree) is

```

current ← tree.root
if current = Nil then
    @ no min
else
    while [current].left ≠ Nil ex
        current ← [current].left
    minimum ← [currentNode].info

```

$O(m)$

parent

function parent (tree, node) is:

```

c ← tree.root
if c = node then
    parent ← Nil
else
    while c ≠ Nil and [c].left ≠ node and [c].right ≠ node ex
        if [c].info ≥ [node].info then
            c ← [c].left
        else
            c ← [c].right
    parent ← c

```

↑ OR
a whole node

$O(m)$

Successor

function $scc(tree, node)$ is

```

if [node].right  $\neq$  Nil then
    c  $\leftarrow$  [node].right
    while [c].left  $\neq$  Nil ex
        c  $\leftarrow$  [c].left
    scc  $\leftarrow$  c

```

else

```

p  $\leftarrow$  parent(tree, c)
while p  $\neq$  Nil and [p].left  $\neq$  c execute
    c  $\leftarrow$  p
    p  $\leftarrow$  parent(tree, p)
scc  $\leftarrow$  p

```

go right once,
then left as
much as possible

why can't we just go
left once?!

?!

→ complexity depends on parent fct.

parent $\Theta(1) \Rightarrow scc$
 $O(n)$

parent $O(n) \Rightarrow scc$
 $O(n^2)$

* ~ efficientize *

→ only last parent.

list → remove from position ✓

→ return from position ✓

→ insert to position X

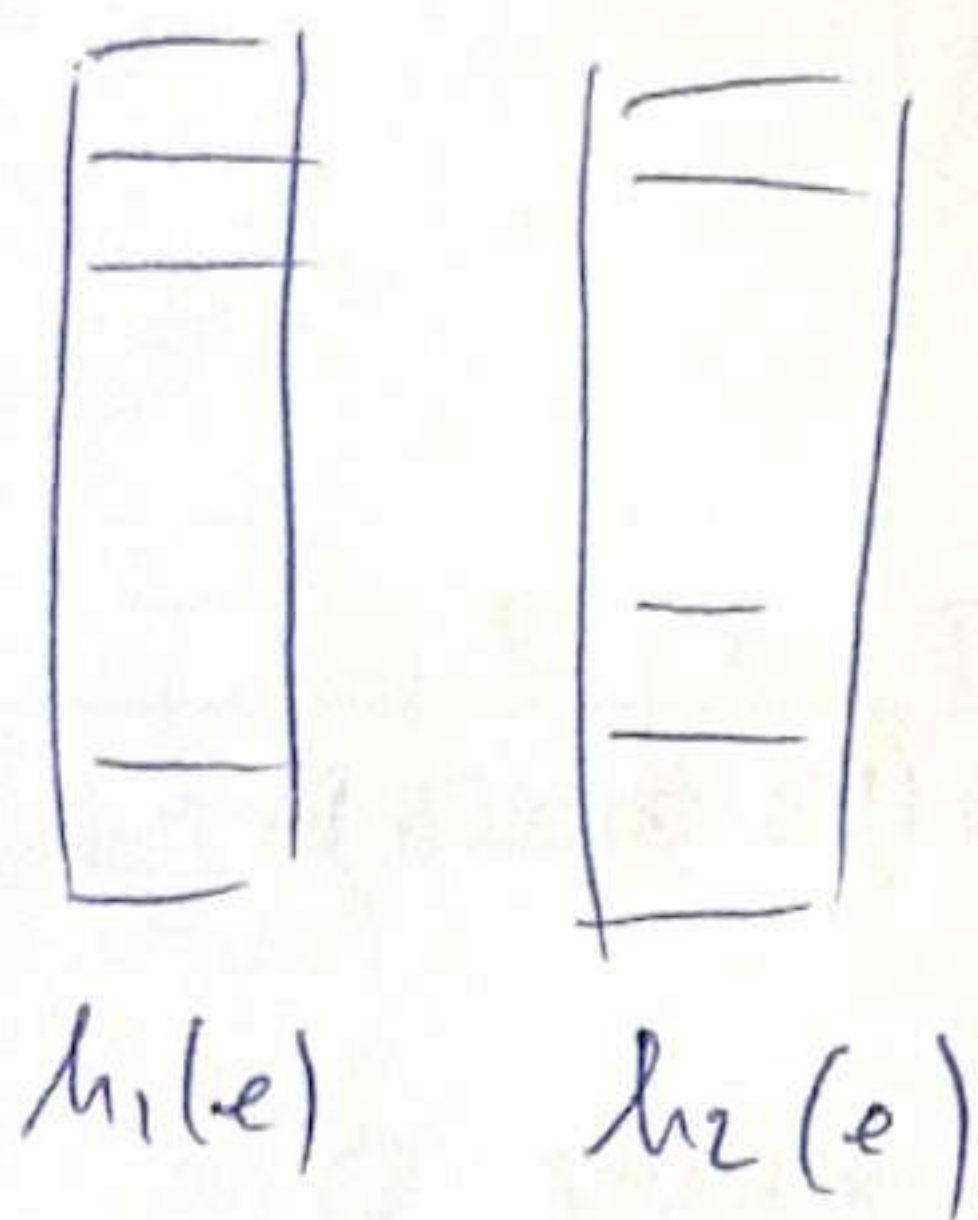
BST → positions → every node keeps

④ nr. nodes in the L
subtree.

(root)

Lecture 11

Cuckoo



$m=11$

H_1	Position	0	1	2	3	4	5	6	7	8	9	10
	T		100					50			20	
			67								53	
											75	

H_2	Position	0	1	2	3	4	5	6	7	8	9	10
	T		20								100	

$$h_1(k) = k \% 11$$

$$h_2(k) = (k / 11) \% 11$$

- Insert 20: $h_1(20) = 9$ ✓

50: $h_1(50) = 6$ ✓

53: $h_1(53) = 9$

↳ 53 in H_1

↳ 20 in $H_2 \rightarrow h_2(20) = (20 / 11) \% 11 = 1$

75: $h_1(75) = 9$

↳ 75 in H_1

$$h_2(75) = 4$$

100: $h_1(100) = 1$

67: $h_1(67) = 1$

↳ $h_2(100) = 9$

cycle $\Rightarrow$ resize + rehash.

if $\alpha < 0.5 \Rightarrow$ low chances of cycles

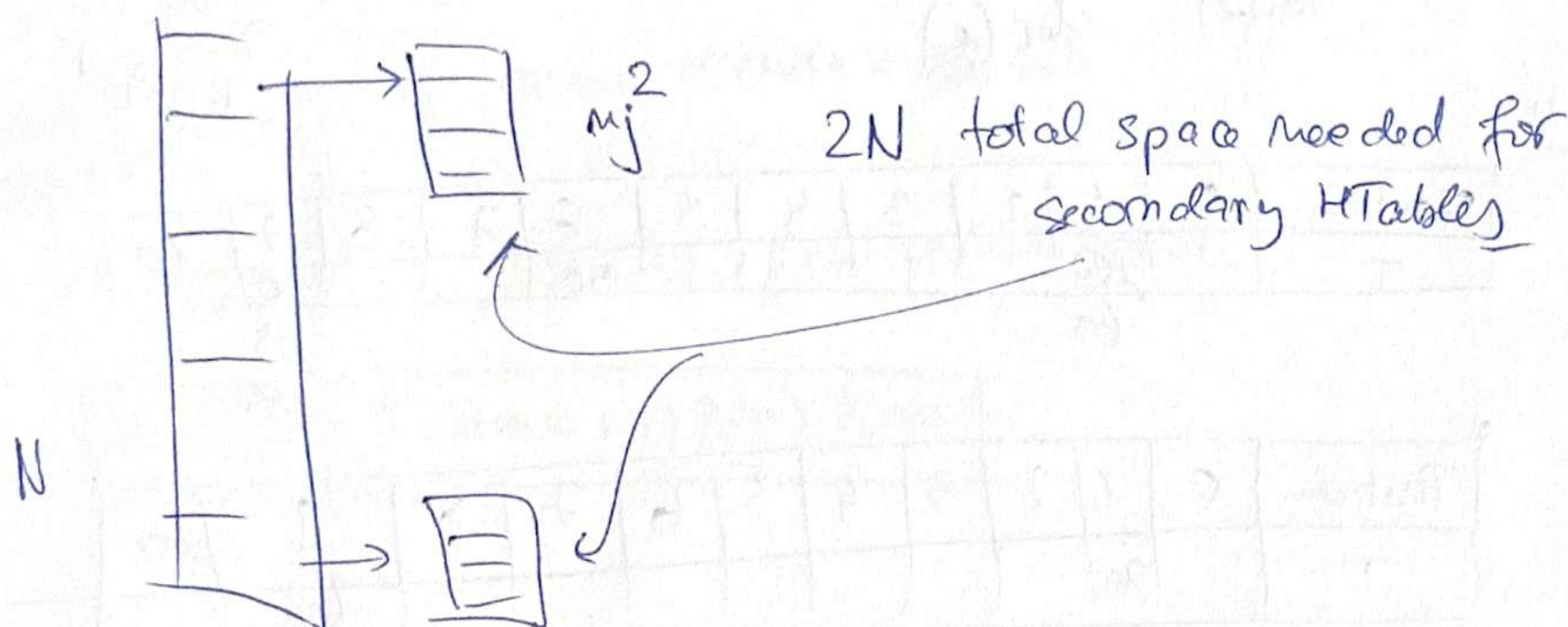
unlikely that more than $O(\log_2 n)$ elems will be moved

$\alpha < 0.5$. (both)

Perfect hashing

- Know all keys in advance
 $H = N^2$ impractical.

→ collision $m \Rightarrow$ new hash fct. (2 trials)



> Hash functions $\Rightarrow$ Universal hashing.

p prime nr.

$$\mathcal{H} = \{ H_{a,b}(x) = ((a \cdot x + b) \% p) \% m \}$$

$$\mathcal{H} = \{ H_{a,b}(x) = ((a \cdot x + b) \% p) \% m \}$$

$$\begin{aligned} a &\in [1, p-1] \\ b &\in [0, p-1] \end{aligned} \quad \left. \begin{array}{l} \\ \end{array} \right\} \text{random!}$$

PERFECT HASHING EXAMPLE

↳ unique letters: PERFECTHASINGXHL

$$N = \text{size} = 15 = m.$$

Letter	P	E	R	F	C	T	H	A	S	I	N	G	X	M	L
HashCode	16	5	18	6	3	20	8	1	19	9	14	7	24	13	12

index
alphabet

$$p \text{ prime} > \text{max} = 24$$

$$p = 29 \text{ (first prime } > 24)$$

$$H_{a,b}(x) = ((a * x + b) \% p) \% m$$

$$a=3, b=2$$

Letter	P
HashCode	16
H(HashCode)	6

$$H_{3,2}(16) = ((3 * 16 + 2) \% 29) \% 15$$

$$= (50 \% 29) \% 15$$

$$= 21 \% 15$$

$$= 6$$

$$\begin{array}{r} 16 \\ \times 3 \\ \hline 48 \\ 50 \\ -29 \\ \hline 21 \end{array}$$

⇒ position 6: P → no collision → secondary HT with only one element

⇒ position 0: I, N.

$$h(x) = 0$$

2 elems ⇒ secondary HT with 4 positions

$$a=4, b=11 \text{ hashCode}$$

$$h(I) = h(9) = 2$$

$$h(N) = h(14) = 1$$

pos 1: $a=5, b=2$

$$h(s) = \dots = 2$$

$$h(x) = \dots = 2$$

$\Rightarrow$ choose other: $a=2, b=13$

$$h(s) = 2$$

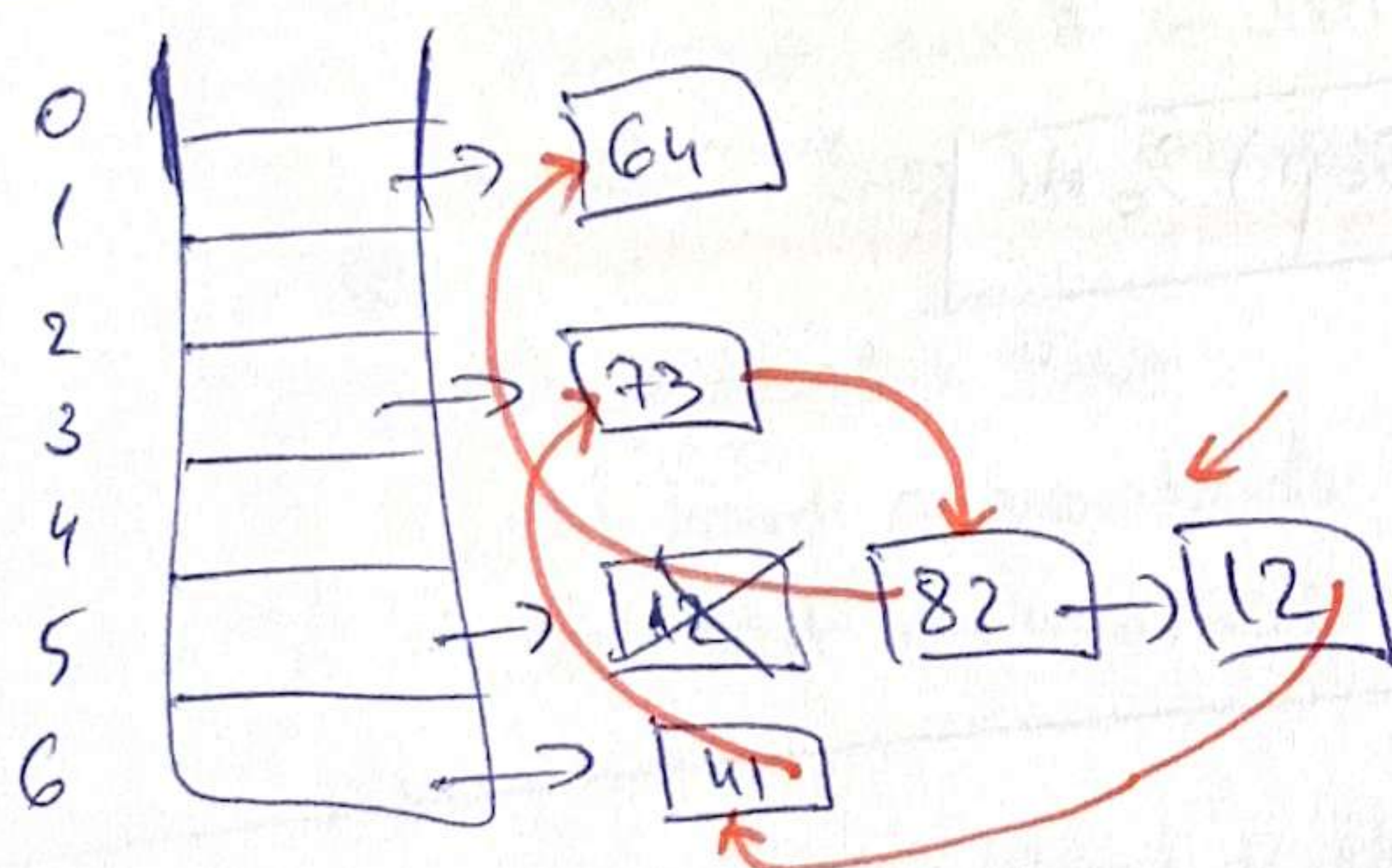
$$h(x) = 3$$

$$u_i: \Theta(1)$$

search for elem. in perf. hashing $\rightarrow$ check at most 2 pos.

STATIC KEYS. (no add)

Linked HT $\rightarrow$ predictable iteration order (insertion order)



insert: 12, 41, 73, 82, 64 $\Rightarrow h(12)$

$$m=7$$

$$h(12)=5$$

$$h(41)=6$$

$$h(73)=3$$

$$h(82)=5 \text{ Beginning}$$

$$h(64)=1$$

HT $\Rightarrow \Theta(1)$ { insert
remove $\rightarrow$ need DU (need previous)
search

Node

into: TKey

nextH: $\uparrow$ Node

nextL: $\uparrow$ Node

prevL: $\uparrow$ Node

Linked HT

m: Int

T: ($\uparrow$ Node)[]

h: TFunction

head: $\uparrow$ Node

tail: $\uparrow$ Node

insert

subalg insert (lht, k) is:

allocate (newNode)

[newNode].info $\leftarrow$ k

[newNode].nextH $\leftarrow$ Nil

• nextL $\leftarrow$ Nil

• prevL $\leftarrow$ Nil

pos $\leftarrow$ lht.h(k)

if lht.T[pos] = Nil then

lht.T[pos] $\leftarrow$ newNode

else

[newNode].nextH $\leftarrow$ lht.T[pos]

lht.T[pos] $\leftarrow$ newNode

if lht.head = Nil then

lht.head $\leftarrow$ newNode

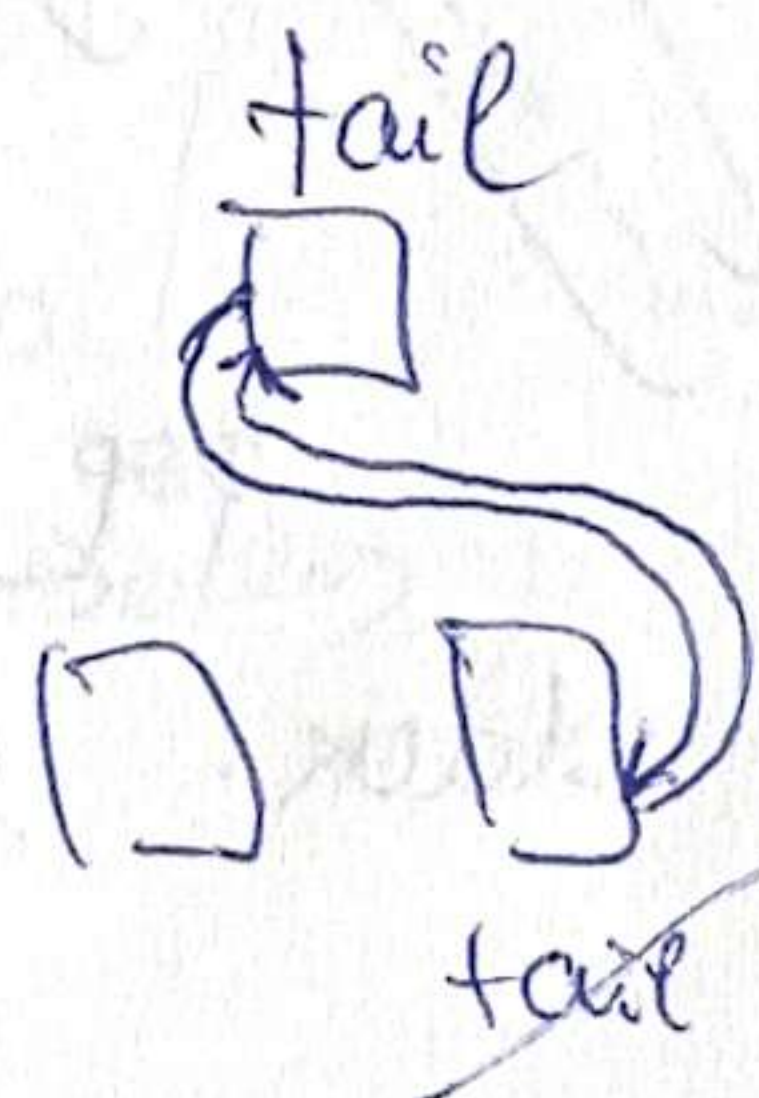
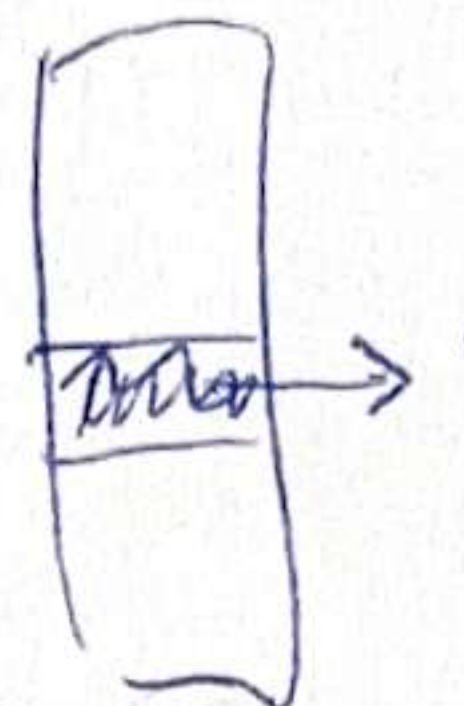
lht.tail $\leftarrow$ newNode

else

[newNode].prevL $\leftarrow$ lht.tail

[lht.tail].nextL $\leftarrow$ newNode

lht.tail $\leftarrow$ newNode



remove

subalg remove (lht, k) is:

pos $\leftarrow$ lht.h(k)

current $\leftarrow$ lht.T[pos]

nodeToRemove $\leftarrow$ Nil

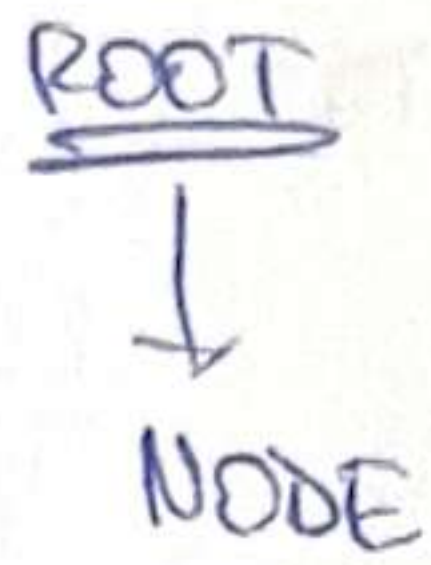
if current $\neq$ Nil and [current].info = k then

nodeToRemove $\leftarrow$ current

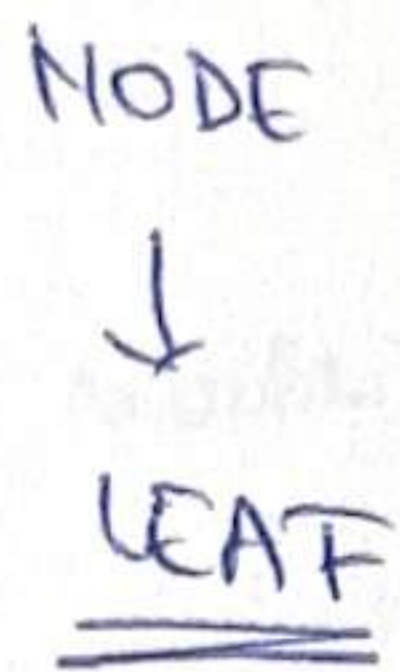
lht.T[pos] $\leftarrow$ [current].nextH

else

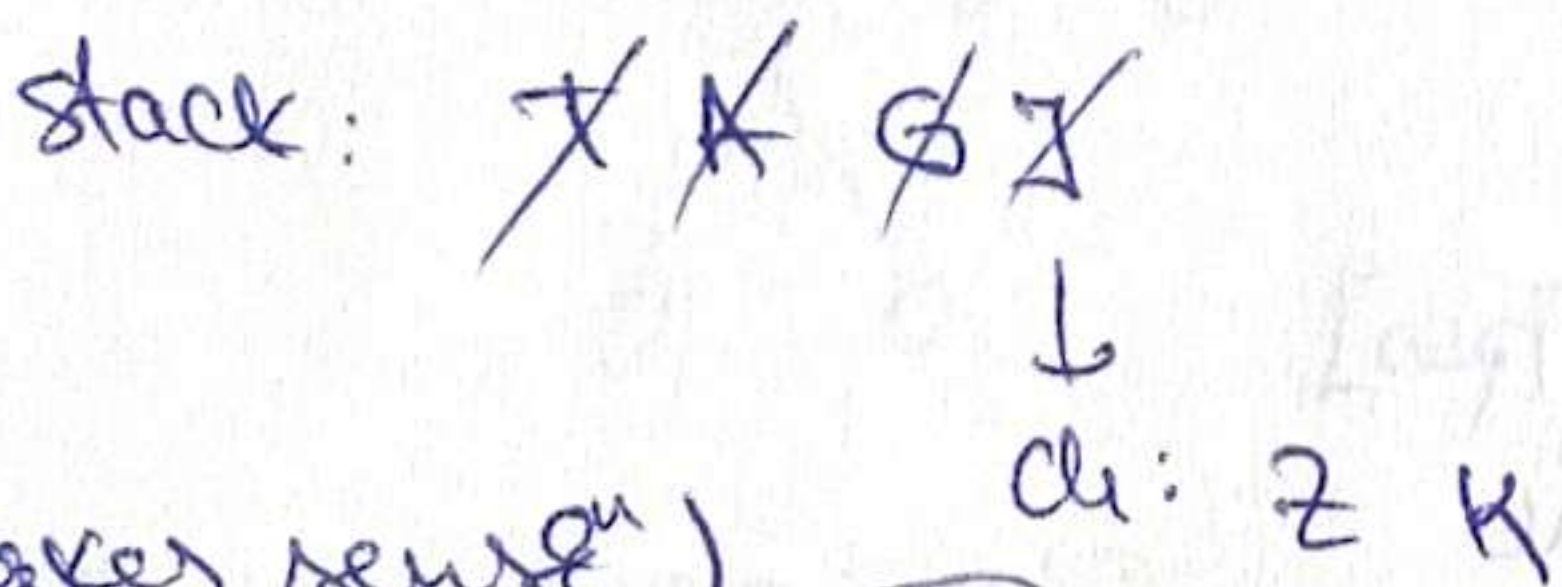
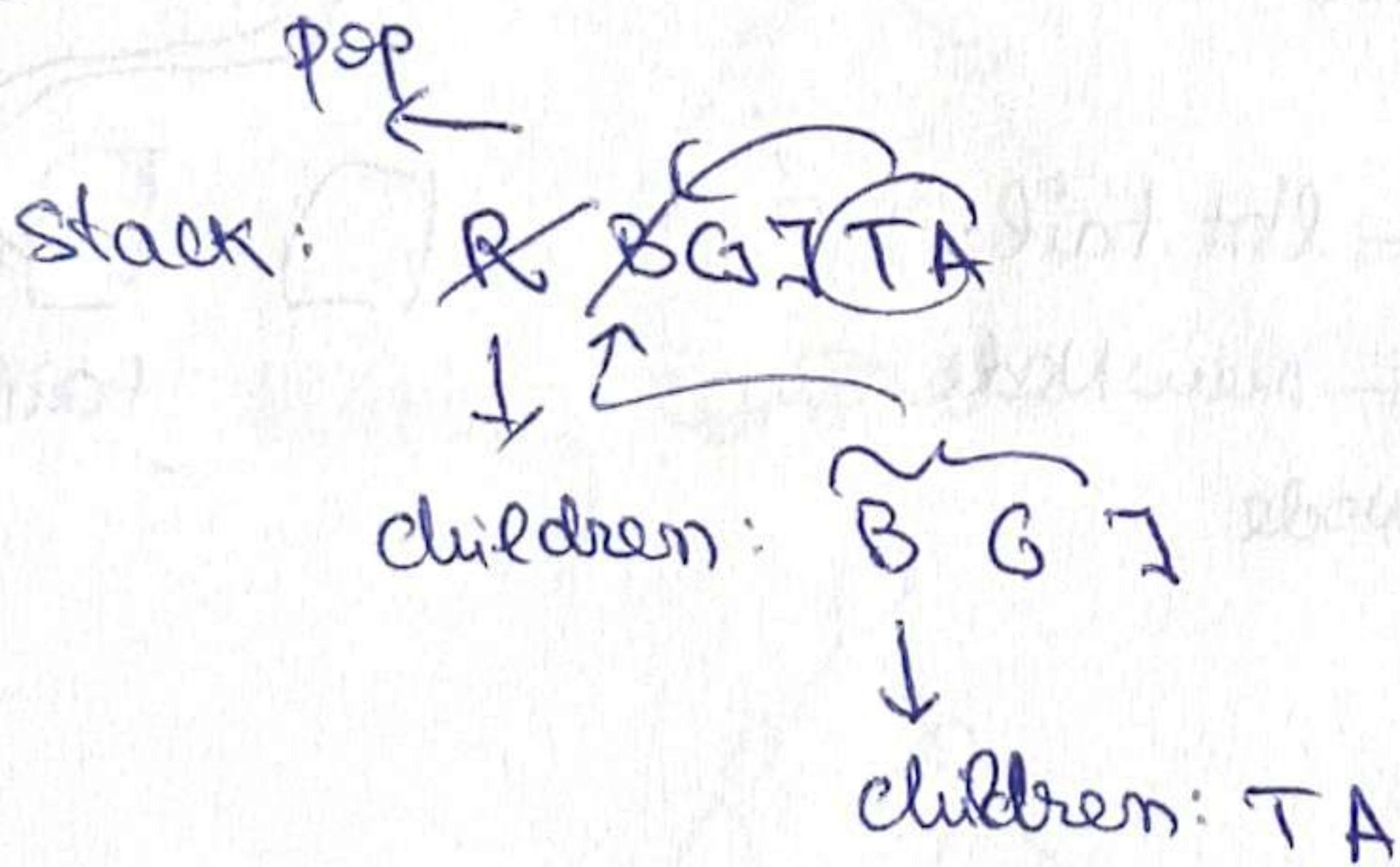
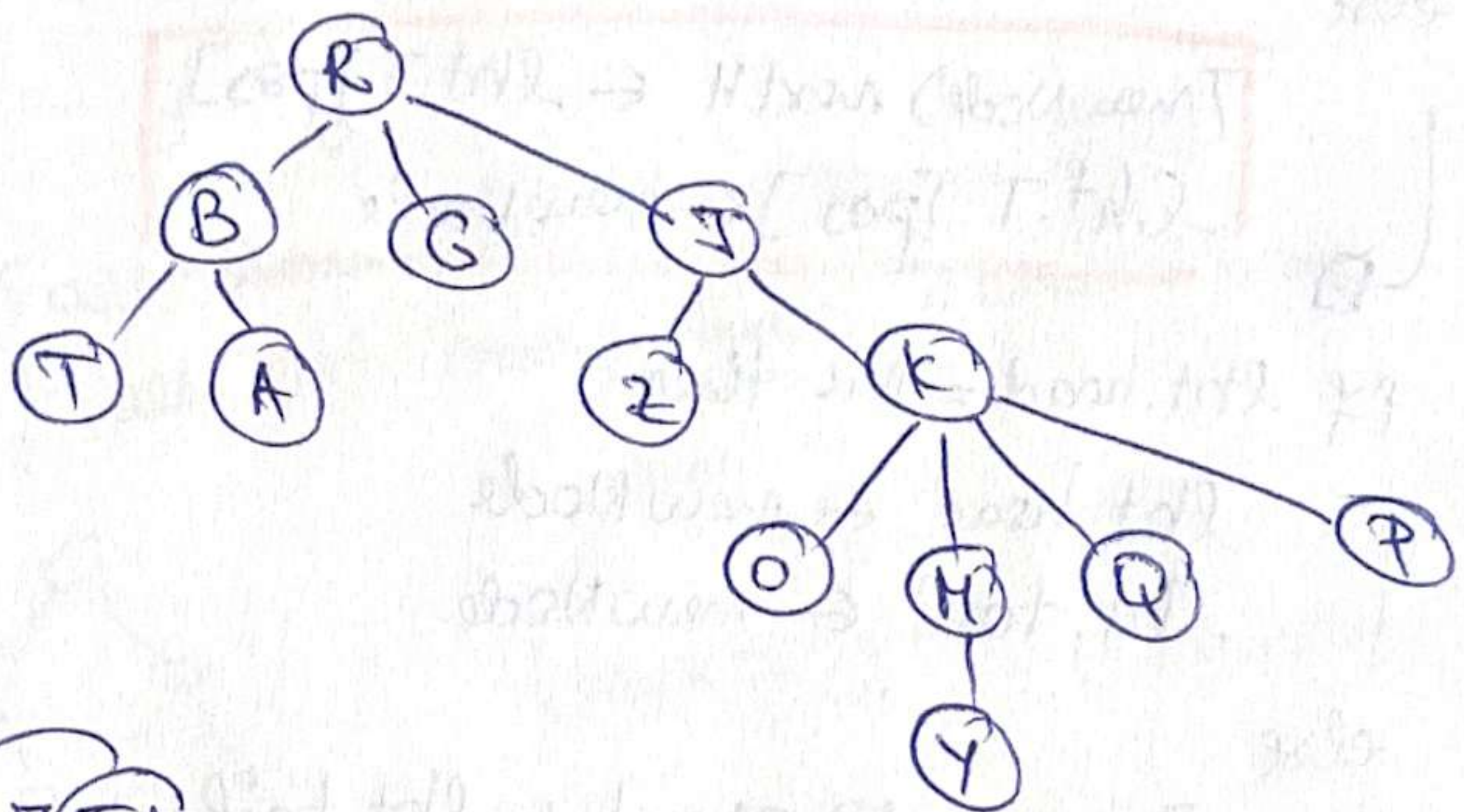
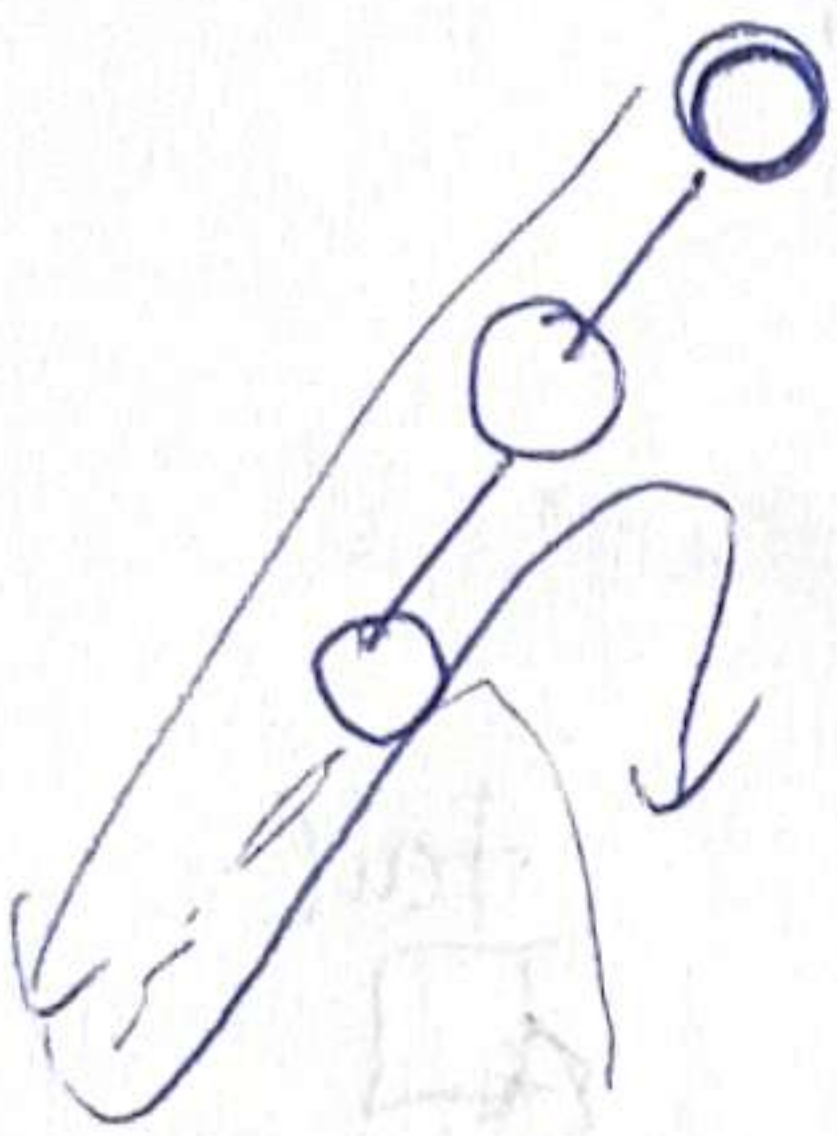
Depth / level:



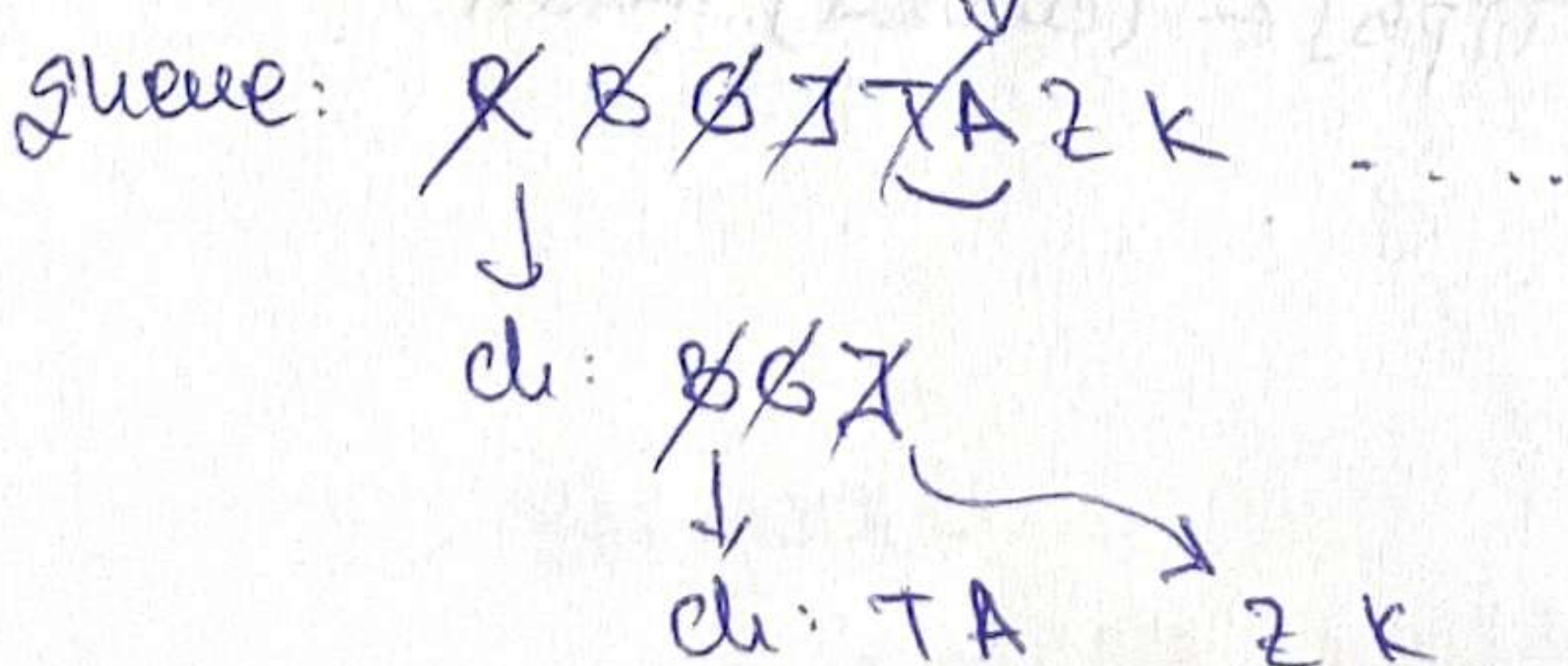
Height:



DFS ⇒ stack

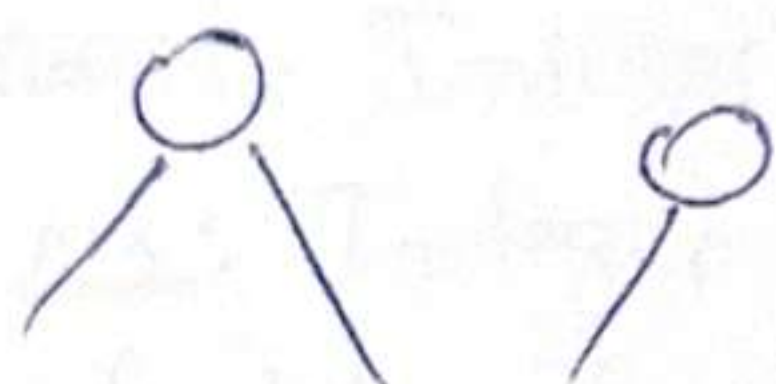


(what makes sense)
Level order (BFS) ⇒ queue



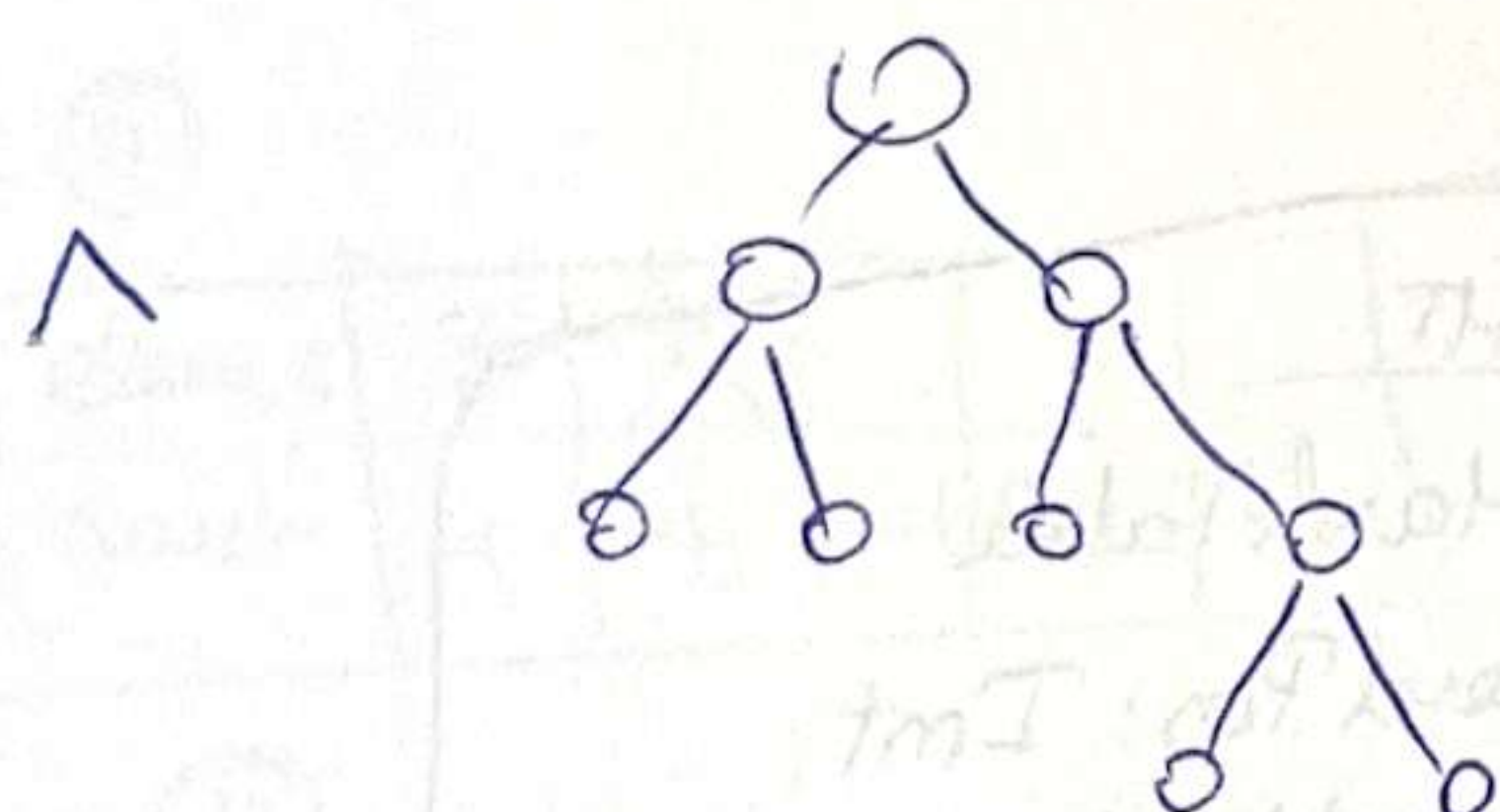
BT

BT with n nodes $\Rightarrow n-1$ edges. (if tree)

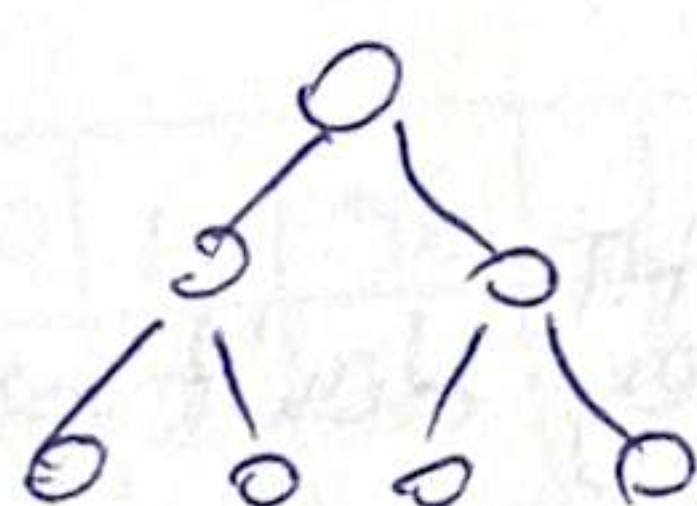


①

FULL $\rightarrow$ (if) int. node has 2 children.

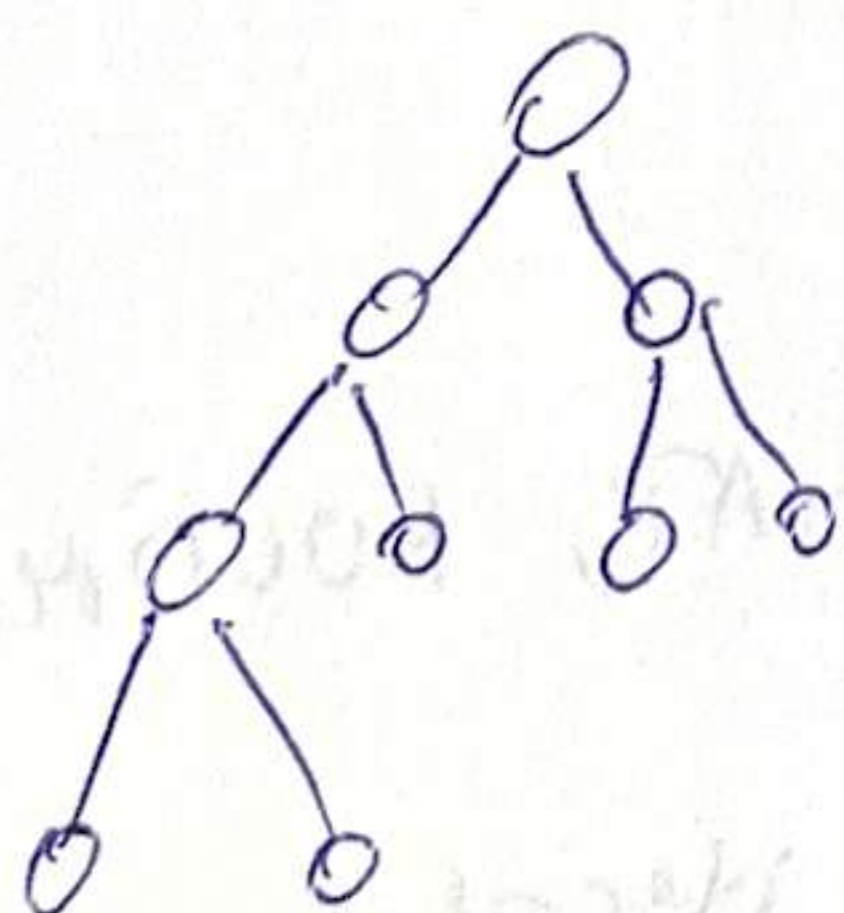


COMPLETE $\rightarrow$ { all leaves on the same level
all int. nodes have 2 children.



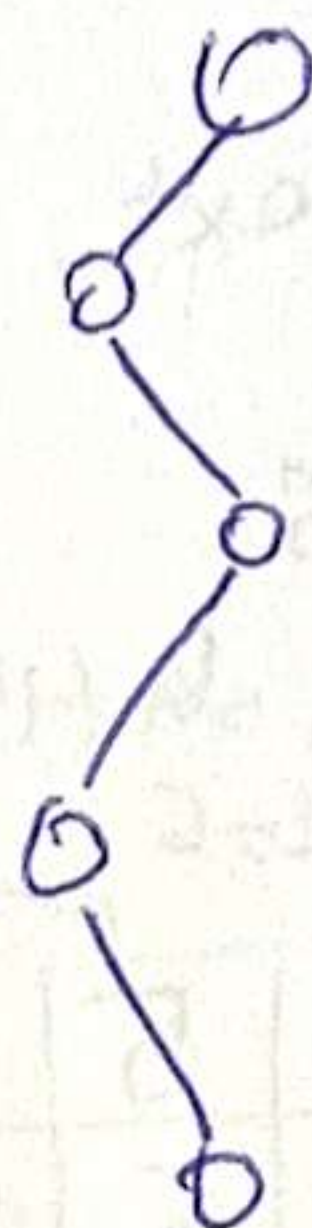
$2^{N+1} - 1$ nodes
(max $= 2^{N+1} - 1$ nodes)

ALMOST COMPLETE (heap)



$h \in (\log_2 N, N-1)$

what $\rightarrow$ "DEGENERATE"
(suzanna likes)



$N+1 = \min$

BALANCED

$(h_{\text{left}} - h_{\text{right}}) \leq 1$

(not AVL)

init(bt) empty

initLeaf(bt, e) ○

initTree(bt, left, e, right) 

Lecture 10

Collision res. w sep. chaining

Node:

key: TKey

next: ↑Node

HT:

T: ↑Node []

m: Integer

h: TFunction

Iterator HT

ht: HashTable

currentPos: Int

currentNode: ↑Node

SEP. CHAINING

HT → NO. Queue
Stack
ordered list
Sorted set / map
... order
(they don't preserve order)

YES: SET, BAG, MAP, MULTIMAP.

BUT: keep sorted lists. + iterator

coalesced no ll, but next. ($\alpha \leq 1$)

insert: 5, 18, 16, 15, 13, 31, 26.

$h(5)=5, h(18)=5, h(16)=3, h(15)=2, h(13)=0, h(31)=5, h(26)=0$
 $13 \rightarrow FE=1, 26 \rightarrow FE=6, 18 \rightarrow FE=0, 31 \rightarrow FE=4$

pos	0	1	2	3	4	5	6	7	8	9	10	11	12
T	18	13	15	16	31	5	26						
next	1	2	-	-	-	5	-	-	-	-	-	-	-

firstEmpty: ~~0~~ ~~1~~ ~~2~~ ~~6~~ ~~7~~